مقاله پژوهشی دانشگاهی · سیستم‌عامل

زمان‌بند کاملاً منصفانه (CFS): طراحی، پیاده‌سازی، عملکرد و مقایسه با الگوریتم‌های زمان‌بندی بی‌درنگ

مطالعه‌ای جامع در باب زمان‌بندی CPU در سیستم‌عامل های نهفته

نام و نام خانوادگی دانشجو

محمدرضا عضدی

نام دانشگاه

دانشگاه اشراق

نام درس

سیستم های نهفته

تاریخ تحویل

۱۴۰۵

نام استاد / مدرس

استاد مجید شمشیرگران

دانشکده / گروه

کامپیوتر

نیمسال تحصیلی

اول

چکیده

زمان‌بندی CPU یکی از کارکردهای محوری هر سیستم‌عامل همه‌منظوره است. این کارکرد تعیین می‌کند که کدام وظیفه آماده اجرا، چه مدت و روی کدام هسته پردازنده زمان دریافت کند و از این رو به‌طور مستقیم بر گذردهی، تأخیر، انصاف و پیش‌بینی‌پذیری اثر می‌گذارد. این مقاله مطالعه‌ای جامع درباره **زمان‌بند کاملاً منصفانه (CFS)** ارائه می‌دهد؛ کلاس زمان‌بندی منصفانه‌ای که لینوکس از نسخه هسته ۲٫۶٫۲۳ تا معرفی تدریجی الگوریتم **اولین مهلت مجازی واجد شرایط (EEVDF)** در لینوکس ۶٫۶، آن را به‌عنوان زمان‌بند پیش‌فرض رومیزی و سرور خود به کار می‌برد. این مقاله مدل مفهومی CFS — یعنی «CPU ایده‌آل و دقیق چندوظیفگی» که از طریق زمان اجرای مجازی (`vruntime`)، وزن وظایف برگرفته از مقدار nice و درخت قرمز-سیاه مرتب‌شده بر اساس زمان تقریب زده می‌شود — را توضیح می‌دهد و سپس جریان کاری عملی، شبه‌کد، توازن بار چندپردازنده‌ای، زمان‌بندی گروهی و محدودیت‌های این طراحی را بررسی می‌کند. فصلی جداگانه CFS را با الگوریتم‌های کلاسیک بی‌درنگ **نرخ یکنوا (RM)** و **اولین مهلت (EDF)** همراه با کران‌های زمان‌بندی‌پذیری و مفروضات آن‌ها مقایسه می‌کند. مقاله تأکید می‌کند که CFS یک زمان‌بند همه‌منظوره با جهت‌گیری انصاف است و تضمین بی‌درنگ سخت ارائه نمی‌دهد. همچنین CFS کلاسیک را از کلاس زمان‌بندی منصفانه مبتنی بر EEVDF در نسخه‌های جدیدتر تفکیک کرده و توضیح می‌دهد که چرا آگاهی از نسخه هسته برای تفسیر رفتار زمان‌بندی لینوکس ضروری است. مثال‌های گویا، نمودارهای SVG، جدول‌ها و یک نمایش عملی قابل بازتولید در لینوکس نیز گنجانده شده است. تمام مثال‌های کمی به‌صراحت به‌عنوان نمونه‌های آموزشی و نه اندازه‌گیری‌های محک معرفی شده‌اند.

واژگان کلیدی

زمان‌بندی CPU · زمان‌بند کاملاً منصفانه · CFS · EEVDF · هسته لینوکس · زمان اجرای مجازی · درخت قرمز-سیاه · انصاف نسبی · نرخ یکنوا · اولین مهلت · زمان‌بندی بی‌درنگ · چند هسته‌ای · توازن بار · cgroups

فهرست مطالب

چکیده i

واژگان کلیدی i

فهرست شکل‌ها ii

فهرست جدول‌ها iii

فهرست اختصارات iv

۱. مقدمه ۱

- ۱٫۱ نقش زمان‌بندی CPU ۱
- ۱٫۲ اهمیت زمان‌بندی در محیط‌های چندوظیفگی ۲
- ۱٫۳ انصاف، گذردهی، پاسخ‌دهی و پیش‌بینی‌پذیری ۲
- ۱٫۴ تحول تاریخی زمان‌بندی CPU در لینوکس ۳
- ۱٫۵ انگیزه پشت CFS ۳
- ۱٫۶ اهداف، دامنه و روش‌شناسی پژوهش ۴
- ۱٫۷ سازماندهی مقاله ۴

۲. مبانی زمان‌بندی CPU ۵

- ۲٫۱ فرایندها، رشته‌ها، وظایف و انفجارهای CPU ۵
- ۲٫۲ زمان‌بندی پیشگیرانه در برابر غیرپیشگیرانه ۵
- ۲٫۳ صف‌های زمان‌بندی و تعویض زمینه ۶
- ۲٫۴ معیارهای عملکرد و مثال‌های حل‌شده ۶
- ۲٫۵ بارهای کاری تعاملی در برابر CPU-محور ۸
- ۲٫۶ زمان‌بندی همه‌منظوره در برابر بی‌درنگ ۸
- ۲٫۷ مبانی زمان‌بندی SMP و چند هسته‌ای ۹

۳. زمان‌بند کاملاً منصفانه (CFS) ۱۰

- ۳٫۱ چندوظیفگی ایده‌آل ۱۰
- ۳٫۲ انصاف نسبی ۱۰
- ۳٫۳ زمان اجرای مجازی (`vruntime`) ۱۱
- ۳٫۴ وزن وظایف و اولویت زمان‌بندی ۱۱
- ۳٫۵ مقادیر nice و تخصیص CPU ۱۲
- ۳٫۶ موجودیت‌های زمان‌بندی ۱۲
- ۳٫۷ ساختار وظایف آماده اجرا و درخت قرمز-سیاه ۱۳
- ۳٫۸ انتخاب وظیفه بعدی ۱۴
- ۳٫۹ به‌روزرسانی زمان اجرای مجازی ۱۴
- ۳٫۱۰ پیش‌دستی و تصمیم‌های زمان‌بندی ۱۵

۳٫۱۱ ایجاد، پایان، انسداد و بیدار شدن وظیفه ۱۵
۳٫۱۲ توازن بار و وابستگی CPU ۱۶
۳٫۱۳ زمان‌بندی گروهی و انصاف سلسله‌مراتبی ۱۶
۳٫۱۴ CFS و کلاس‌های زمان‌بندی لینوکس ۱۷
۳٫۱۵ CFS کلاسیک در برابر EEVDF ۱۷
۴. جریان کاری الگوریتم CFS ۱۹
۴٫۱ گام‌به‌گام جریان کاری ۱۹
۴٫۲ نمودار جریان لازم ۲۰
۴٫۳ مدل مفهومی در برابر پیاده‌سازی ۲۱
۵. شبه‌کد و مثال حل‌شده CFS ۲۲
۵٫۱ شبه‌کد مفهومی ۲۲
۵٫۲ مثال حل‌شده با سه وظیفه ۲۳
۵٫۳ تخصیص انتظاری در برابر مشاهده‌شده ۲۵
۶. ویژگی‌ها و مزایای اصلی CFS ۲۷
۷. زمان‌بندی چندپردازنده و چند هسته‌ای ۳۰
۸. ارزیابی عملکرد و نمودارها ۳۳
۹. مقایسه CFS، RM و EDF ۳۷
۱۰. محدودیت‌ها و معایب CFS ۴۱
۱۱. گرسنگی، بن‌بست و وارونگی اولویت ۴۳
۱۲. امنیت، قابلیت اطمینان و ملاحظات عملی ۴۶
۱۳. نمایش عملی در لینوکس ۴۸
۱۴. بحث و جهت‌های آینده ۵۰
۱۵. نتیجه‌گیری ۵۲
منابع ۵۳

فهرست شکل‌ها

شکل ۳٫۱ — رابطه مفهومی میان زمان اجرای واقعی، وزن و زمان اجرای مجازی ص. ۱۲

شکل ۳٫۲ — درخت قرمز-سیاه ساده‌شده از موجودیت‌های زمان‌بندی آماده اجرا کلیدخورده بر اساس `vruntime` ص. ۱۳

شکل ۴٫۱ — نمودار جریان جریان کاری زمان‌بندی CFS ص. ۲۰

شکل ۵٫۱ — نمودار گانت برای مثال حل‌شده سه‌وظیفه‌ای ص. ۲۵

شکل ۷٫۱ — صف‌های اجرای هر CPU و مهاجرت وظیفه در یک سیستم SMP ص. ۳۱

شکل ۸٫۱ — تخصیص گویای CPU میان وظایف با مقادیر nice متفاوت ص. ۳۴

شکل ۸٫۲ — تحول گویای زمان اجرای مجازی برای سه وظیفه ص. ۳۴

شکل ۸٫۳ — نمودار گانت بازه‌های اجرای گویای CFS ص. ۳۵

شکل ۸٫۴ — مقایسه کیفی ویژگی‌های زمان‌بندی RM، CFS و EDF ص. ۳۵

شکل ۸٫۵ — توازن بار میان چهار هسته CPU ص. ۳۶

فهرست جدول‌ها

جدول ۲٫۱ — مقایسه زمان‌بندی پیشگیرانه و غیرپیشگیرانه ص. ۵

جدول ۲٫۲ — معیارهای زمان‌بندی و تعاریف آن‌ها ص. ۷

جدول ۳٫۱ — مفاهیم و اصطلاحات اصلی CFS ص. ۱۱

جدول ۳٫۲ — مقادیر nice، وزن‌ها و سهم‌های نسبی CPU به‌صورت گویا ص. ۱۲

جدول ۵٫۱ — پارامترهای اولیه وظایف برای مثال حل‌شده ص. ۲۳

جدول ۵٫۲ — تحول زمان اجرای مجازی در مثال حل‌شده ص. ۲۴

جدول ۶٫۱ — مزایا و معایب CFS ص. ۲۸

جدول ۹٫۱ — مقایسه تفصیلی CFS، RM و EDF ص. ۳۸

جدول ۱۱٫۱ — گرسنگی در برابر بن‌بست در برابر وارونگی اولویت ص. ۴۵

جدول ۱۳٫۱ — ابزارهای زمان‌بندی لینوکس و کاربردهای آن‌ها ص. ۴۸

جدول ۳٫۳ — CFS کلاسیک در برابر مفاهیم جدیدتر مرتبط با EEVDF ص. ۱۸

جدول ۱۵٫۱ — خلاصه یافته‌های اصلی ص. ۵۲

فهرست اختصارات

اختصار	معنا
CFS	زمان‌بند کاملاً منصفانه
EEVDF	اولین مهلت مجازی واجد شرایط
RM	زمان‌بندی نرخ یکنوا
EDF	زمان‌بندی اولین مهلت
SMP	چندپردازشی متقارن
NUMA	دسترسی نامتقارن به حافظه
vruntime	زمان اجرای مجازی
VRT	زمان اجرای مجازی (افت تأخیر EEVDF)
VD	مهلت مجازی (EEVDF)
RT	بی‌درنگ
SCHED_NORMAL	سیاست زمان‌بندی منصفانه پیش‌فرض لینوکس
SCHED_FIFO	سیاست بی‌درنگ FIFO در POSIX
SCHED_RR	سیاست بی‌درنگ نوبت گردشی در POSIX
SCHED_DEADLINE	سیاست زمان‌بندی مهلت‌محور لینوکس
cgroup	گروه کنترل لینوکس
PELT	ردیابی بار هر موجودیت

۱. مقدمه

۱٫۱ نقش زمان‌بندی CPU

یک سیستم‌عامل مسئول مالتی‌پلکس کردن مجموعه‌ای محدود از پردازنده‌های فیزیکی میان مجموعه‌ای بالقوه بزرگ و پویا از وظایف آماده اجراست. مؤلفه‌ای که تصمیم لحظه‌به‌لحظه «کدام وظیفه بعدی اجرا شود» را می‌گیرد، **زمان‌بند CPU** نامیده می‌شود. زمان‌بندی یک دغدغه حاشیه‌ای نیست؛ بلکه در تقاطع عملکرد، انصاف، پاسخ‌دهی، مصرف انرژی و ایزولاسیون قرار دارد. زمان‌بندی که پیوسته به نفع وظایف طولانی CPU-محور عمل کند ممکن است گذردهی را بیشینه سازد اما رومیزی تعاملی را غیرپاسخگو کند؛ زمان‌بندی که به‌شدت به نفع وظایف حساس به تأخیر باشد ممکن است پاسخ‌دهی ادراک‌شده را بهبود بخشد، به بهای گذردهی و انصاف. طراحی یک زمان‌بند همه‌منظوره بنابراین تمرینی در بهینه‌سازی مقید تحت شرایط بار کاری نامعلوم و ناهمگن است.

در لینوکس، زمان‌بندی به‌صورت سلسله‌مراتبی از **کلاس‌های زمان‌بندی** پیاده‌سازی می‌شود. هر کلاس یک سیاست را کپسوله می‌کند و توزیع‌کننده هسته کلاس‌ها را به ترتیب اولویت فرامی‌خواند تا وظیفه‌ای آماده اجرا بیابد. **کلاس زمان‌بندی منصفانه** وظایف معمول کاربر — یعنی آن‌هایی که از `SCHED_NORMAL`، `SCHED_BATCH` و `SCHED_IDLE` استفاده می‌کنند — را مدیریت می‌کند و برای بیش از پانزده سال، الگوریتم اصلی آن **زمان‌بند کاملاً منصفانه (CFS)** بود. CFS در لینوکس ۲٫۶٫۲۳ ادغام شد و توسط اینگو مولنار به‌عنوان جانشین زمان‌بند پیشین `SCHED_OTHER` طراحی گردید. هدف اصلی آن تقریب زدن یک «CPU ایده‌آل و دقیق چندوظیفگی» روی سخت‌افزار واقعی بود. بنابراین درک CFS مستلزم درک هم مدل مفهومی ظریف آن و هم سازش‌های مهندسی است که آن را عملی می‌سازند.

۱٫۲ اهمیت زمان‌بندی در محیط‌های چندوظیفگی

سیستم‌های مدرن به‌طور معمول صدها یا هزاران رشته را اجرا می‌کنند. حتی یک جلسه رومیزی متوسط ممکن است دارای compositor مرورگر، سرور صوتی، ایندکسر فایل، کامپایلر و چندین دیمن پس‌زمینه باشد که برای چند هسته رقابت می‌کنند. زمان‌بند تعیین می‌کند که آیا کاربر پیمایش نرم را ادراک می‌کند، آیا بافر صوتی زیرجریان می‌رود، آیا ساخت به‌موقع کامل می‌شود و آیا یک کار پس‌زمینه می‌تواند ماشین را در انحصار بگیرد. از آن‌جا که بارهای کاری انفجاری و ناهمگن هستند، هیچ تخصیص اولویت ثابتی نمی‌تواند برای همه فازهای اجرا بهینه باشد. یک زمان‌بند باید پیوسته باز-ارزیابی کند که کدام وظیفه باید اجرا شود، چه مدت و روی کدام CPU. این تصمیم‌ها با محلیت حافظه نهان، پهنای باند حافظه، مدیریت توان و همگام‌سازی تعامل دارند و به همین دلیل زمان‌بندی با وجود دهه‌ها مطالعه همچنان حوزه‌ای فعال در پژوهش سیستم‌هاست.

۱٫۳ انصاف، گذردهی، پاسخ‌دهی و پیش‌بینی‌پذیری

چهار هدف برای ارزیابی یک زمان‌بند همه‌منظوره به‌کار می‌رود و اغلب با یکدیگر در تنش هستند:

- **انصاف** به چگونگی توزیع زمان CPU میان وظایف آماده اجرا مربوط است. یک زمان‌بند منصف می‌کوشد به هر وظیفه سهمی متناسب با استحقاقش بدهد، نه این‌که یک وظیفه پردازنده را در انحصار بگیرد.
- **گذردهی** مقدار کار مفید کامل‌شده در واحد زمان را می‌سنجد. سیاست‌های گذردهی‌محور ممکن است به نفع انفجارهای طولانی CPU و کاهش سربار تعویض زمینه عمل کنند.

- **پاسخ‌دهی** سرعت واکنش یک وظیفه به رویداد خارجی را می‌سنجد. بارهای کاری تعاملی و حساس به تأخیر از تأخیر زمان‌بندی کوتاه سود می‌برند، حتی اگر این گذردهی خام را کاهش دهد.
- **پیش‌بینی‌پذیری** به واریانس زمان پاسخ و زمان تکمیل مربوط است. سیستم‌های بی‌درنگ به تأخیر بدترین حالت کران‌دار نیاز دارند، در حالی که سیستم‌های همه‌منظوره معمولاً تأخیر میانگین پایین و تأخیر دنباله قابل‌قبول را هدف می‌گیرند.

CFS صراحتاً یک زمان‌بند *انصاف‌محور* است. این الگوریتم تضمین مهلت سخت نمی‌دهد و رفتار آن نباید با رفتار یک زمان‌بند بی‌درنگ اشتباه گرفته شود. این تمایز در فصل‌های ۹ و ۱۰ بسط داده می‌شود.

۱٫۴ تحول تاریخی زمان‌بندی CPU در لینوکس

زمان‌بندی لینوکس از چندین مرحله متمایز گذشته است. هسته‌های اولیه از یک صف اجرای دایره‌ای ساده با تابع goodness استفاده می‌کردند که اولویت و زمان‌برش باقی‌مانده را ترکیب می‌کرد. **زمان‌بند O(1)** که در سری ۲٫۶ معرفی شد، از آرایه‌های اولویت هر CPU استفاده می‌کرد و انتخاب با زمان ثابت را ممکن می‌ساخت، اما اکتشافی‌های آن برای تعاملی‌بودن به‌سختی قابل تنظیم شدند و انصاف ناسازگاری تولید کردند. **زمان‌بند نوبت گردشی پله‌ای (RSDL)** و پیشین آن **SD** ایده‌های سهم منصفانه را پیش از ادغام CFS در ۲٫۶٫۲۳ کاویدند. CFS مفهوم زمان‌برش ثابت را با یک ساعت مجازی و درخت قرمز-سیاه جایگزین کرد و حسابداری انصاف را صریح ساخت و بسیاری از اکتشافی‌های ad hoc را حذف کرد. در لینوکس ۶٫۶ که در اکتبر ۲۰۲۳ منتشر شد، کلاس منصفانه شروع به گذار به **EEVDF** کرد؛ الگوریتمی که نخست در سال ۱۹۹۵ توسط استویکا و عبدالوهاب پیشنهاد و توسط پیتر زیجلسترا برای لینوکس اقتباس شد. این گذار مهم است: گزاره‌هایی درباره «زمان‌بند لینوکس» که برای هسته x.۵ معتبر بودند ممکن است برای هسته ۶٫۶ و پس از آن نادرست باشند.

۱٫۵ انگیزه پشت CFS

انگیزه بی‌واسطه CFS، انصاف درک‌شده و پیچیدگی اکتشافی زمان‌بند O(1) بود، به‌ویژه تحت بارهای کاری ترکیبی تعاملی و CPU-محور. زمان‌بند O(1) می‌کوشید با مشاهده رفتار خواب، تعاملی‌بودن یک وظیفه را استنتاج و اولویت آن را متناسباً تنظیم کند. این اکتشافی‌ها شکننده بودند: یک وظیفه می‌توانست بد-طبقه‌بندی شود و تنظیمات اولویت حاصل می‌توانست انحرافات بزرگ انصاف تولید کند. CFS رویکرد دیگری در پیش گرفت. به‌جای برآورد تعاملی‌بودن، پردازنده‌ای ایده‌آل را مدل می‌کند که می‌تواند همه وظایف آماده اجرا را هم‌زمان و با سرعت برابر اجرا کند و سپس ردگیری می‌کند که هر وظیفه چه مقدار زمان مجازی مصرف کرده است. وظیفه‌ای که کمترین زمان اجرای مجازی را دارد بعدی انتخاب می‌شود. این قاعده واحد یک معیار انصاف منسجم ارائه می‌دهد، به‌طور طبیعی اولویت را از طریق وزن‌دهی جای می‌دهد و از بسیاری از آسیب‌شناسی‌های اکتشافی‌های پیشین پرهیز می‌کند.

۱٫۶ اهداف، دامنه و روش‌شناسی پژوهش

این مقاله پنج هدف دارد: (الف) توضیح مبانی مفهومی CFS، شامل چندوظیفگی ایده‌آل، زمان اجرای مجازی، وزن‌ها و درخت قرمز-سیاه؛ (ب) توصیف جریان کاری الگوریتم، شبه‌کد و یک مثال حل‌شده؛ (ج) تحلیل زمان‌بندی چندپردازنده و چند هسته‌ای، شامل توازن بار، وابستگی و ملاحظات NUMA؛ (د) مقایسه CFS با الگوریتم‌های کلاسیک بی‌درنگ RM و EDF؛ و (ه) شناسایی محدودیت‌های CFS و اهمیت گذار CFS به EEVDF. روش‌شناسی تحلیلی و اسنادی است. این

مقاله بر مستندات رسمی هسته لینوکس، مستندات کد منبع هسته، ادبیات بنیادین زمان‌بندی و صفحات راهنمای معتبر تکیه دارد. مثال‌های کمی برای اهداف آموزشی ساخته شده‌اند و صراحتاً گویا برچسب خورده‌اند؛ هیچ ادعایی مبنی بر این‌که آن‌ها اندازه‌گیری‌های محک از یک ساخت هسته مشخص را بازتولید می‌کنند مطرح نمی‌شود.

۱٫۷ سازماندهی مقاله

فصل ۲ مبانی زمان‌بندی و معیارها را مرور می‌کند. فصل ۳ مدل CFS را به‌تفصیل ارائه می‌دهد، شامل ساختار داده‌های آن و رابطه‌اش با کلاس‌های زمان‌بندی. فصل ۴ جریان کاری الگوریتم را توصیف و یک نمودار جریان ارائه می‌کند. فصل ۵ شبه‌کد مستقل از زبان و یک مثال حل‌شده کامل ارائه می‌دهد. فصل ۶ ویژگی‌ها و مزایای اصلی CFS را ارزیابی می‌کند. فصل ۷ زمان‌بندی چندپردازنده و چند هسته‌ای را بررسی می‌کند. فصل ۸ نمودارهای گویا ارائه و مفروضات آن‌ها را توضیح می‌دهد. فصل ۹ RM، CFS و EDF را مقایسه می‌کند. فصل ۱۰ محدودیت‌ها را بحث می‌کند. فصل ۱۱ به گرسنگی، بن‌بست و وارونگی اولویت می‌پردازد. فصل ۱۲ امنیت، قابلیت اطمینان و پیکربندی عملی را پوشش می‌دهد. فصل ۱۳ یک نمایش عملی قابل بازتولید در لینوکس ارائه می‌دهد. فصل ۱۴ جهت‌های آینده را بحث می‌کند و فصل ۱۵ نتیجه‌گیری می‌کند.

۲. مبانی زمان‌بندی CPU

۲٫۱ فرایندها، رشته‌ها، وظایف و انفجارهای CPU

در لینوکس، زمان‌بند بر روی **وظایف** (که با `struct task_struct` نمایش داده می‌شوند) عمل می‌کند؛ این وظایف بسته به اشتراک فضای آدرس و منابع دیگر ممکن است متناظر با فرایند یا رشته باشند. یک وظیفه میان **انفجارهای CPU** و **انفجارهای I/O** نوسان می‌کند. وظایف CPU-محور انفجارهای CPU طولانی و I/O کم‌تواتر دارند؛ وظایف I/O-محور و تعاملی انفجارهای CPU کوتاه و انسداد مکرر دارند. توزیع طول انفجارها وابسته به بار کاری و اغلب دم-سنگین است. یک زمان‌بند نمی‌تواند طول انفجارهای آینده را بداند، بنابراین باید تصمیم‌های آنلاین با استفاده از رفتار مشاهده‌شده و پارامترهای اعلام‌شده مانند مقدار nice و سیاست زمان‌بندی اتخاذ کند. CFS وظایف را به تعاملی یا CPU-محور طبقه‌بندی نمی‌کند؛ در عوض از طریق زمان اجرای مجازی، اجرای واقعی را حساب می‌کند و به این ترتیب می‌تواند به تقاضای مشاهده‌شده پاسخ دهد نه به قصد استنتاج‌شده.

۲٫۲ زمان‌بندی پیشگیرانه در برابر غیرپیشگیرانه

یک زمان‌بند **غیرپیشگیرانه** به وظیفه در حال اجرا اجازه می‌دهد تا زمانی که به‌طور داوطلبانه مسدود شود، خاتمه یابد یا CPU را واگذار کند، به اجرا ادامه دهد. یک زمان‌بند **پیشگیرانه** می‌تواند وظیفه در حال اجرا را قطع کند وقتی وظیفه‌ای با اولویت بالاتر آماده اجرا می‌شود یا وقتی برش زمانی‌اش منقضی می‌گردد. پیش‌دستی برای پاسخ‌دهی و انصاف در سیستم‌های همه‌منظوره ضروری است، اما سربار تعویض زمینه را معرفی می‌کند و می‌تواند محلیت حافظه نهان را مختل کند. لینوکس در هسته پیشگیرانه است (با برخی استثناها در بخش‌های اتمی) و CFS برای اجرای اشتراک منصفانه بر پیش‌دستی تکیه دارد. جدول ۲٫۱ سازش‌ها را خلاصه می‌کند.

جدول ۲٫۱ — مقایسه زمان‌بندی پیشگیرانه و غیرپیشگیرانه.

معیار	پیشگیرانه	غیرپیشگیرانه
نقاط تصمیم	ورود، بیدار شدن، تیک، انسداد، واگذاری	انسداد، پایان، واگذاری داوطلبانه
پاسخ‌دهی	معمولاً بهتر؛ یک وظیفه با اولویت بالاترِ تازه آماده اجرا می‌تواند پیش‌دستی کند	احتمالاً ضعیف؛ یک وظیفه طولانی می‌تواند دیگران را به تأخیر بیندازد
انصاف	اجرای آن با برش زمانی آسان‌تر است	نیازمند همکاری؛ برای وظایف کوتاه ناعادلانه است
سربار	بیشتر: تعویض‌های زمینه و فراخوانی‌های زمان‌بند	کمتر: تعویض‌های کمتر
رفتار حافظه نهان	اختلال بیشتر به‌دلیل مهاجرت و تعویض	محلیت بهتر برای وظایف طولانی
کاربرد معمول	سیستم‌عامل همه‌منظوره، سیستم‌عامل بی‌درنگ	سیستم‌های دسته‌ای، برخی سیستم‌های نهفته

۲٫۳ صف‌های زمان‌بندی و تعویض زمینه

وظایف آماده اجرا در **صف‌های اجرا** سازمان می‌یابند. در CFS، هر CPU دارای ساختار `cfs_rq` است که درخت قرمز-سیاهی از موجودیت‌های زمان‌بندی آماده اجرا مرتب‌شده بر اساس `vruntime` را در خود دارد. وقتی زمان‌بند از وظیفه‌ای به وظیفه دیگر جابه‌جا می‌شود، یک **تعویض زمینه** انجام می‌دهد: وضعیت ثبات وظیفه خارج‌شونده را ذخیره و وضعیت وظیفه داخل‌شونده را بازیابی می‌کند. تعویض زمینه رایگان نیست؛ چرخه مصرف می‌کند، حافظه‌های نهان و بافرهای جستجوی ترجمه (TLB) را آلوده می‌کند و می‌تواند با چندرشته‌ای هم‌زمان تعامل ضعیفی داشته باشد. CFS می‌کوشد تعویض‌های غیرضروری را با استفاده از یک گرانولاریتی بیدار شدن کاهش دهد تا از پیش‌دستی فوری وظیفه تازه بیدارشده بر وظیفه جاری جلوگیری کند، مگر این‌که اختلاف زمان اجرای مجازی به‌اندازه کافی بزرگ باشد. این یک سازش عامدانه میان انصاف و سربار است.

۲٫۴ معیارهای عملکرد و مثال‌های حل‌شده

تصمیم‌های زمان‌بندی با معیارهای استاندارد ارزیابی می‌شوند. فرض کنید n تعداد وظایف، T_i زمان تکمیل وظیفه i، A_i زمان ورود آن، B_i کل نیاز انفجار CPU آن و W_i زمان انتظار آن باشد. آن‌گاه:

$$Turnaround_i = T_i - A_i \tag{۲٫۱}$$

$$Waiting_i = Turnaround_i - B_i \tag{۲٫۲}$$

$$Response_i = FirstRun_i - A_i \tag{۲٫۳}$$

$$Throughput = \frac{\text{number of completed tasks}}{\text{total elapsed time}} \tag{۲٫۴}$$

$$CPU\ utilization = \frac{\text{busy time}}{\text{total time}} \tag{۲٫۵}$$

به‌عنوان یک مثال ساده، سه وظیفه را در نظر بگیرید که در زمان صفر وارد می‌شوند و نیازهای انفجاری آن‌ها $B_1 = ۸$، $B_2 = ۴$ و $B_3 = ۲$ واحد زمانی است. تحت یک سیاست نخستین‌ورود-نخستین‌سرویس غیرپیشگیرانه به ترتیب ۱، ۲، ۳، زمان‌های تکمیل ۸، ۱۲ و ۱۴ است، لذا میانگین زمان چرخش (۸ + ۱۲ + ۱۴)/۳ = ۱۱٫۳۳ و میانگین زمان انتظار (۰ + ۸ + ۱۲)/۳ = ۶٫۶۷ است. اگر وظیفه ۳ ابتدا، سپس وظیفه ۲ و سپس وظیفه ۱ اجرا شود، زمان‌های تکمیل ۲، ۶ و ۱۴، میانگین زمان چرخش ۷٫۳۳ و میانگین زمان انتظار ۲٫۶۷ خواهد بود. این مثال کلاسیک نشان می‌دهد که ترتیب زمان‌بندی حتی وقتی کل کار تغییر نمی‌کند، به‌طور مادی بر تأخیر ادراک‌شده کاربر اثر می‌گذارد. CFS به‌طور مستقیم میانگین زمان چرخش را بهینه نمی‌کند؛ هدف آن انصاف نسبی است، اما مثال نشان می‌دهد که چرا سیاست زمان‌بندی اهمیت دارد.

جدول ۲٫۲ – معیارهای زمان‌بندی و تعاریف آن‌ها.

معیار	تعریف	دغدغه اصلی
بهره‌وری CPU	کسری از زمان که CPU کار مفید انجام می‌دهد	کارایی
گذردهی	وظایف تکمیل‌شده در واحد زمان	کارایی
زمان چرخش	زمان تکمیل منهای زمان ورود	عملکرد دسته‌ای
زمان انتظار	زمان سپری‌شده در صف اجرا	انصاف / دسته‌ای
زمان پاسخ	زمان از ورود تا نخستین اجرا	تعاملی‌بودن
انصاف	انحراف از سهم استحقاقی CPU	اشتراک نسبی
سربار	زمان سپری‌شده در زمان‌بند و تعویض‌های زمینه	کارایی

۲٫۵ بارهای کاری تعاملی در برابر CPU-محور

بارهای کاری تعاملی — ویرایشگر متن، ترمینال، مرورگر، سرور صوتی — به‌طور معمول انفجارهای CPU کوتاه و سپس انسداد روی I/O یا ورودی کاربر دارند. عملکرد ادراک‌شده آن‌ها به‌شدت به **زمان پاسخ** و **تأخیر زمان‌بندی** بستگی دارد. بارهای کاری CPU-محور — کامپایلرها، رمزگذارهای ویدئو، شبیه‌سازی‌های علمی — انفجارهای CPU طولانی دارند و از **گذردهی** و محلیت حافظه نهان سود می‌برند. یک زمان‌بند همه‌منظوره باید هر دو کلاس را بدون طبقه‌بندی صریح سرویس دهد. حسابداری زمان اجرای مجازی CFS به‌طور طبیعی به وظیفه‌ای که مدت‌ها خوابیده زمان اجرای مجازی نسبتاً کوچکی می‌دهد (با تبعیت از قواعد جای‌گذاری)، بنابراین می‌تواند به‌موقع در هنگام بیدار شدن زمان‌بندی شود، در حالی که یک وظیفه CPU-محور که مقدار زیادی CPU مصرف کرده زمان اجرای مجازی انباشته می‌کند و به دیگران واگذار می‌کند. این یک تضمین سخت تأخیر نیست، بلکه راهی اصولی برای توازن میان دو نوع بار کاری است.

۲٫۶ زمان‌بندی همه‌منظوره در برابر بی‌درنگ

زمان‌بندی همه‌منظوره هدفش ارائه رفتار میانگین خوب در بارهای کاری متنوع است. زمان‌بندی بی‌درنگ هدفش تضمین این است که هر وظیفه مهلت خود را حتی در بدترین حالت رعایت کند. این اهداف متفاوت‌اند و می‌توانند تعارض داشته باشند. یک زمان‌بند بی‌درنگ سخت ممکن است انصاف و گذردهی را فدای رعایت مهلت کند؛ یک زمان‌بند منصف ممکن است یک وظیفه حساس به مهلت را به نفع انصاف به تأخیر بیندازد. لینوکس هر دو جهان را از طریق کلاس‌های زمان‌بندی ارائه می‌دهد. کلاس منصفانه (CFS و بعدها EEVDF) وظایف معمولی را مدیریت می‌کند. کلاس بی‌درنگ `SCHED_FIFO` و `SCHED_RR` را با اولویت‌های ۱ تا ۹۹ پیاده‌سازی می‌کند و کلاس مهلت `SCHED_DEADLINE` را بر پایه EDF با کنترل پذیرش پیاده می‌کند. وظایف بی‌درنگ همیشه بر وظایف کلاس منصفانه پیش‌دستی می‌کنند. خود CFS تضمین بی‌درنگ سخت ارائه نمی‌دهد و این تمایز هنگام مقایسه CFS با RM و EDF در فصل ۹ ضروری است.

۲٫۷ مبانی زمان‌بندی SMP و چند هسته‌ای

در یک سیستم چندپردازشی متقارن (SMP)، هر CPU صف اجرای خود را دارد و زمان‌بند نه‌تنها باید تصمیم بگیرد کدام وظیفه اجرا شود بلکه روی کدام CPU. این امر **توازن بار**، **مهاجرت وظیفه**، **وابستگی CPU** و **محلیت حافظه نهان** را به دغدغه‌های درجه‌یک تبدیل می‌کند. مهاجرت یک وظیفه به یک CPU بیکار ممکن است بهره‌وری را بهبود دهد اما

می‌تواند وضعیت حافظه نهان و TLB را از بین ببرد. لینوکس توپولوژی CPU را به‌صورت سلسله‌مراتبی از **دامنه‌های زمان‌بندی** که از مرزهای حافظه نهان و NUMA ساخته شده‌اند مدل می‌کند و توازن بار می‌کوشد کار را در داخل دامنه‌ها پیش از عبور از آن‌ها جابه‌جا کند. این موضوع‌ها در فصل ۷ به‌تفصیل بررسی می‌شوند.

۳. زمان‌بند کاملاً منصفانه (CFS)

۳٫۱ چندوظیفگی ایده‌آل

مدل مفهومی محوری CFS، **CPU ایده‌آل و دقیق چندوظیفگی** است. پردازنده‌ای با توان ۱۰۰٪ را تصور کنید که می‌تواند همه وظایف آماده اجرا را هم‌زمان، هر یک با سرعت $1/n$ اجرا کند، که n تعداد وظایف آماده اجراست. اگر دو وظیفه آماده اجرا باشند، هر یک ۵۰٪ پردازنده را دریافت می‌کند؛ اگر چهار وظیفه باشند، هر یک ۲۵٪. این ماشین ایده‌آل تأخیر زمان‌بندی و سربار تعویض زمینه ندارد. سخت‌افزار واقعی تنها می‌تواند تعداد محدودی وظیفه را هم‌زمان اجرا کند، بنابراین CFS با ردگیری این‌که هر وظیفه چه مقدار زمان مجازی در ماشین ایده‌آل دریافت می‌کرد و با اجرای همیشگی وظیفه‌ای که بیشترین عقب‌ماندگی را دارد، ایده‌آل را تقریب می‌زند. این تقریب جوهر طراحی است: همان‌طور که مستندات هسته می‌گویند، ۸۰٪ طراحی CFS را می‌توان در این جمله خلاصه کرد که CFS یک CPU چندوظیفگی ایده‌آل را روی سخت‌افزار واقعی مدل می‌کند.

۳٫۲ انصاف نسبی

اشتراک برابر صرف برای یک سیستم همه‌منظوره کافی نیست، زیرا کاربران و مدیران باید بتوانند وزن‌های متفاوتی به وظایف اختصاص دهند. بنابراین CFS **انصاف نسبی** را پیاده‌سازی می‌کند: هر وظیفه مستحق سهمی از زمان CPU متناسب با وزن خود است. اگر وظایف A و B وزن‌های w_A و w_B داشته باشند، در یک سیستم اشباع، سهم‌های بلندمدت CPU باید برآورده کنند:

$$\frac{CPU_A}{CPU_B} \approx \frac{w_A}{w_B} \tag{۳٫۱}$$

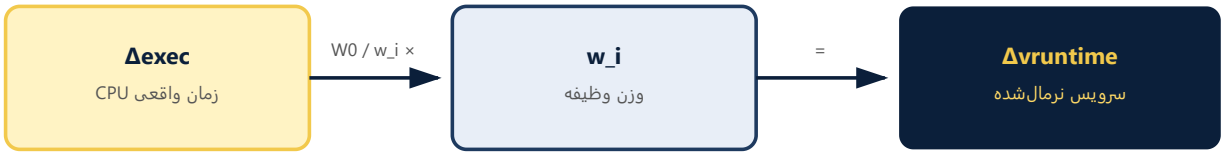
این یک گزاره درباره نسبت‌های بلندمدت تحت رقابت پایدار است. تضمینی درباره هیچ بازه کوتاهی نیست و فرض می‌کند که همه وظایف در تمام بازه آماده اجرا هستند. در عمل، وظایف مسدود می‌شوند، بیدار می‌شوند، مهاجرت می‌کنند و وزن خود را تغییر می‌دهند، بنابراین سهم‌های به‌دست‌آمده تقریبی‌اند. زمان‌بندی گروهی همان ایده را به‌صورت سلسله‌مراتبی گسترش می‌دهد: یک گروه کنترل را می‌توان به‌عنوان یک موجودیت زمان‌بندی با وزن خودش در نظر گرفت و وظایف درون گروه، تخصیص گروه را به اشتراک می‌گذارند.

۳٫۳ زمان اجرای مجازی (`vruntime`)

CFS برای هر موجودیت زمان‌بندی یک **زمان اجرای مجازی** در `p→se.vruntime` (یا فیلد متناظر یک موجودیت زمان‌بندی گروهی) به واحد نانوثانیه نگه می‌دارد. زمان اجرای مجازی، زمان واقعی اجرا نیست. این یک معیار نرمال‌شده از میزان سرویس CPU است که موجودیت نسبت به وزنش دریافت کرده است. وقتی یک موجودیت برای Δ_{exec} نانوثانیه اجرا می‌شود، زمان اجرای مجازی آن به میزانی پیش می‌رود که با وزنش نسبت معکوس دارد:

$$\Delta vruntime = \Delta_{exec} \times \frac{W_0}{w_i} \qquad (۳٫۲)$$

در این‌جا W_0 وزن یک وظیفه مرجع است (وزن متناظر با مقدار nice صفر که به‌طور متعارف در CFS کلاسیک ۱۰۲۴ است) و w_i وزن موجودیت i است. وظیفه‌ای با وزن بزرگ‌تر، زمان اجرای مجازی را کندتر انباشته می‌کند، بنابراین می‌تواند پیش از آن‌که زمان اجرای مجازی‌اش به وظیفه‌ای با وزن کمتر برسد، طولانی‌تر اجرا شود. برعکس، وظیفه‌ای با وزن کوچک‌تر زمان اجرای مجازی را به‌سرعت انباشته می‌کند و زودتر پیش‌دستی خواهد شد. زمان‌بند موجودیتی را با کوچک‌ترین زمان اجرای مجازی انتخاب می‌کند، یعنی موجودیتی که نسبت به استحقاقش کمترین سرویس را دریافت کرده است. شکل ۳٫۱ رابطه مفهومی را نشان می‌دهد.



شکل ۳٫۱ — رابطه مفهومی میان زمان اجرای واقعی، وزن وظیفه و زمان اجرای مجازی. وزن بزرگ‌تر نرخ انباشت زمان اجرای مجازی را کاهش می‌دهد. این معادله یک مدل آموزشی ساده‌شده است؛ هسته‌های عملی تنظیمات اضافی برای زمان‌بندی گروهی، تأخیر و جای‌گذاری اعمال می‌کنند.

۳٫۴ وزن وظایف و اولویت زمان‌بندی

در CFS کلاسیک، وزن هر وظیفه از **مقدار nice** آن مشتق می‌شود؛ یک عدد صحیح در بازه −۲۰ تا +۱۹. مقدار nice صفر پیش‌فرض است و وزن ۱۰۲۴ دارد. هر پله یک سطح nice وزن را با ضریب تقریباً ۱٫۲۵ تغییر می‌دهد که با تغییر حدود ۱۰٪ در سهم CPU تحت رقابت متناظر است. نگاشت دقیق یک جدول جستجو در هسته است. جدول ۳٫۱ مفاهیم کلیدی CFS را خلاصه می‌کند و جدول ۳٫۲ وزن‌های گویا را برای مقادیر منتخب nice ارائه می‌دهد.

جدول ۳٫۱ — مفاهیم و اصطلاحات اصلی CFS.

مفهوم	معنا در CFS
CPU ایده‌آل چندوظیفگی	پردازنده‌ای مفهومی که همه وظایف آماده اجرا را هم‌زمان با سرعت n/1 اجرا می‌کند
`vruntime`	زمان اجرای مجازی نرمال‌شده؛ کلید اصلی زمان‌بندی
وزن	مقدار مشتق‌شده از سطح nice؛ نرخ انباشت `vruntime` را تعیین می‌کند
موجودیت زمان‌بندی	یک وظیفه یا گروهی از وظایف که می‌توانند به‌عنوان یک واحد زمان‌بندی شوند
صف اجرا (`cfs_rq`)	ساختار هر CPU که موجودیت‌های آماده اجرا را نگه می‌دارد
درخت قرمز-سیاه	درخت خودمتوازن مرتب بر اساس `vruntime`؛ چپ‌ترین گره بعدی انتخاب می‌شود
`min_vruntime`	کران پایین یکنوای افزایشی برای جای‌گذاری موجودیت‌های جدید یا بیدارشده
گرانولاریتی بیدار شدن	آستانه‌ای که مانع پیش‌دستی فوری یک وظیفه تازه بیدارشده می‌شود
زمان‌بندی گروهی	اشتراک منصفانه سلسله‌مراتبی از طریق گروه‌های کنترل
توازن بار	مهاجرت موجودیت‌های آماده اجرا میان CPUها

جدول ۳٫۲ — مقادیر nice، وزن‌ها و سهم‌های نسبی CPU به‌صورت گویا (جدول وزن CFS کلاسیک؛ تقریبی).

مقدار nice	وزن	سهم نسبت به nice صفر	معنای کیفی
-۲۰	۸۸,۷۶۱	≈ ×۸۶٫۷	بالاترین اولویت متعارف
-۱۰	۹,۵۴۳	≈ ×۹٫۳	اولویت بالا
-۵	۳,۳۵۰	≈ ×۳٫۳	اولویت نسبتاً بالا
۰	۱,۰۲۴	×۱٫۰	پیش‌فرض
+۵	۳۳۵	≈ ×۰٫۳۳	اولویت نسبتاً پایین
+۱۰	۱۱۰	≈ ×۰٫۱۱	اولویت پایین
+۱۹	۱۵	≈ ×۰٫۰۱۵	پایین‌ترین اولویت متعارف

۳٫۵ مقادیر nice و تخصیص CPU

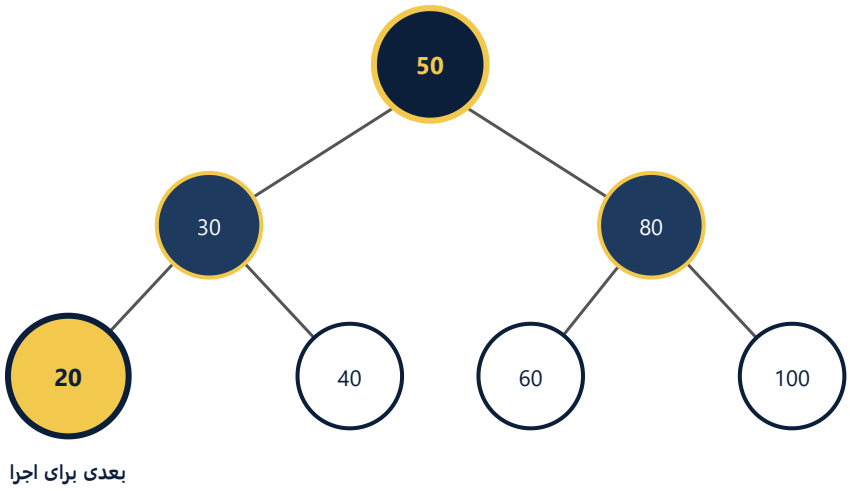
مقدار nice یک نشانه *نسبی* است، نه یک رزرو مطلق. اگر دو وظیفه با مقادیر nice صفر و ۵ تنها وظایف آماده اجرا باشند، نسبت وزن آن‌ها تقریباً ۱۰۲۴:۳۳۵ است، بنابراین وظیفه nice صفر باید حدود ۷۵٪ CPU و وظیفه nice ۵ حدود ۲۵٪ را تحت رقابت پایدار دریافت کند. اگر وظیفه سومی با nice -۵ بپیوندد، نسبت‌ها تغییر می‌کنند. مقادیر nice همچنین بر کلاس‌های زمان‌بندی غلبه نمی‌کنند: یک وظیفه بی‌درنگ با SCHED_FIFO همیشه بر وظیفه کلاس منصفانه پیش‌دستی می‌کند، فارغ از مقدار nice. تغییر مقدار nice یک وظیفه بنابراین با تغییر سیاست زمان‌بندی آن متفاوت است؛ نکته‌ای که در فصل ۱۳ تأکید می‌شود.

۳٫۶ موجودیت‌های زمان‌بندی

CFS **موجودیت‌های زمان‌بندی** را زمان‌بندی می‌کند، نه وظایف خام را. یک موجودیت زمان‌بندی با `struct sched_entity` نمایش داده می‌شود و ممکن است متناظر با یک وظیفه منفرد یا یک گروه وظایف باشد. این انتزاع به CFS امکان می‌دهد گروهی از وظایف را به‌عنوان یک موجودیت وزن‌دار منفرد در نظر بگیرد و انصاف سلسله‌مراتبی را ممکن سازد. هر موجودیت `vruntime`، وزن و سهم بار خود را دارد. وقتی یک گروه انتخاب می‌شود، زمان‌بند به‌صورت بازگشتی وظیفه‌ای درون گروه را با همان الگوریتم منصفانه انتخاب می‌کند. مستندات هسته اشاره می‌کند که CFS ساختارهای `sched_entity` را زمان‌بندی می‌کند، نه وظایف را مستقیماً، که به گروه‌هایی از وظایف امکان می‌دهد شبیه وظایف منفرد زمان‌بندی شوند. زمان‌بندی گروهی در بخش ۳٫۱۳ بیشتر بحث می‌شود.

۳٫۷ ساختار وظایف آماده اجرا و درخت قرمز-سیاه

CFS کلاسیک موجودیت‌های زمان‌بندی آماده اجرا را در یک **درخت قرمز-سیاه** کلیدخورده بر اساس `vruntime` ذخیره می‌کند. درخت قرمز-سیاه یک درخت جستجوی دودویی خودمتوازن است که درج، حذف و جستجو را در O(log n) تضمین می‌کند، در حالی که امکان یافتن کمینه عنصر را در زمان O(1) با دنبال کردن اشاره‌گرهای فرزند چپ فراهم می‌آورد. CFS این ناوردا را حفظ می‌کند که چپ‌ترین گره موجودیتی با کوچک‌ترین زمان اجرای مجازی است، یعنی موجودیت بعدی برای اجرا. صف اجرا همچنین `min_vruntime` را نگه می‌دارد، یک مقدار یکنوای افزایشی که کوچک‌ترین زمان اجرای مجازی مشاهده‌شده در آن صف را ردگیری می‌کند. وقتی یک وظیفه ایجاد یا از خواب بیدار می‌شود، زمان اجرای مجازی آن نسبت به `min_vruntime` جای‌گذاری می‌شود تا پس از یک خواب طولانی به‌طور ناعادلانه به جلوی درخت نپرد. شکل ۳٫۲ یک درخت قرمز-سیاه ساده‌شده را نشان می‌دهد.



هر گره یک موجودیت زمان‌بندی کلیدخورده بر اساس vruntime را نگه می‌دارد (مقادیر گویا).

شکل ۳٫۲ — درخت قرمز-سیاه ساده‌شده از موجودیت‌های زمان‌بندی آماده اجرا کلیدخورده بر اساس `vruntime`. چپ‌ترین گره (۲۰) موجودیت بعدی است که در CFS کلاسیک انتخاب می‌شود. درخت‌های واقعی ممکن است شامل موجودیت‌های گروهی باشند و پس از درج و حذف دوباره متوازن شوند.

۳٫۸ انتخاب وظیفه بعدی

قاعده انتخاب در CFS کلاسیک ساده است: چپ‌ترین موجودیت در درخت قرمز-سیاه را انتخاب کن، یعنی موجودیتی با کوچک‌ترین `vruntime`. اگر درخت خالی باشد، CPU وظیفه بیکار را اجرا می‌کند. از آن‌جا که درخت بر اساس زمان اجرای مجازی مرتب است، گام انتخاب پس از حفظ درخت $O(1)$ است. پیچیدگی در حفظ درخت، به‌روزرسانی زمان اجرای مجازی و تصمیم‌گیری درباره زمان پیش‌دستی نهفته است. در EEVDF انتخاب ظریف‌تر است: زمان‌بند *تأخیر* (این‌که آیا موجودیت مستحق سرویس است یا از آن فراتر رفته) و *مهلت مجازی* (زمانی که تا آن موجودیت باید برش درخواستی خود را دریافت کند) را در نظر می‌گیرد و ممکن است از میان موجودیت‌های واجد شرایط با تأخیر نامنفی، آن را با نزدیک‌ترین مهلت مجازی انتخاب کند. این موضوع در بخش ۳٫۱۵ بحث می‌شود.

۳٫۹ به‌روزرسانی زمان اجرای مجازی

زمان اجرای مجازی در طول اجرا و در رویدادهای زمان‌بندی به‌روزرسانی می‌شود. وقتی موجودیتی برای Δ_{exec} اجرا می‌شود، زمان اجرای مجازی آن مطابق معادله ۳٫۲ افزایش می‌یابد. علاوه بر این، زمان‌بند `min_vruntime` را نگه می‌دارد و ممکن است زمان اجرای مجازی یک موجودیت تازه بیدارشده را طوری تنظیم کند که در نزدیکی سمت چپ درخت جای‌گذاری شود بدون این‌که مزیت ناعادلانه‌ای به دست آورد. منطق به‌روزرسانی دقیق در هسته‌های عملی پیچیده‌تر از معادله ۳٫۲ است: زمان‌بندی گروهی، ردگیری بار (PELT) و در هسته‌های جدیدتر، محاسبات تأخیر و مهلت مجازی را در نظر می‌گیرد. بنابراین معادله ۳٫۲ را باید به‌عنوان یک مدل مفهومی فهمید که جهت و قدر تقریبی اثر را توضیح می‌دهد، نه به‌عنوان توصیفی خط‌به‌خط از `kernel/sched/fair.c`.

۳٫۱۰ پیش‌دستی و تصمیم‌های زمان‌بندی

یک تصمیم زمان‌بندی وقتی رخ می‌دهد که وظیفه‌ای مسدود می‌شود، خاتمه می‌یابد، واگذار می‌کند، بیدار می‌شود، یا وقتی تیک زمان‌بند تشخیص می‌دهد که وظیفه در حال اجرا سهم منصفانه خود را مصرف کرده یا این‌که یک وظیفه تازه آماده اجرا زمان اجرای مجازی به‌اندازه کافی کوچک‌تری دارد. CFS از یک **گرانولاریتی بیدار شدن** برای جلوگیری از پیش‌دستی بیش‌ازحد استفاده می‌کند: یک وظیفه بیدارشونده لزوماً وظیفه جاری را فوراً پیش‌دستی نمی‌کند مگر این‌که اختلاف زمان اجرای مجازی از یک آستانه فراتر رود. این امر تعویض‌های زمینه را کاهش می‌دهد و محلیت حافظه نهان را حفظ می‌کند. پیش‌دستی همچنین تحت تأثیر سلسله‌مراتب کلاس‌های زمان‌بندی است: یک وظیفه بی‌درنگ بیدارشونده فوراً بر وظیفه کلاس منصفانه پیش‌دستی می‌کند، مشروط به محدودسازی بی‌درنگ در صورت پیکربندی.

۳٫۱۱ ایجاد، پایان، انسداد و بیدار شدن وظیفه

وقتی یک وظیفه ایجاد می‌شود، با وزنی مشتق از مقدار nice، یک زمان اجرای مجازی و یک موجودیت زمان‌بندی که در صف اجرای مناسب درج می‌شود مقداردهی می‌گردد. یک وظیفه تازه ایجادشده به‌طور معمول طوری جای‌گذاری می‌شود که سهم بیش‌ازحدی نسبت به وظایف موجود به دست نیاورد. وقتی یک وظیفه مسدود می‌شود، از درخت آماده اجرا حذف و وضعیتش ذخیره می‌گردد. وقتی بیدار می‌شود، با زمان اجرای مجازی تنظیم‌شده نسبت به `min_vruntime` دوباره درج می‌شود؛ این امر مانع از آن می‌شود که یک وظیفه طولانی‌خواب با زمان اجرای مجازی به‌طور مصنوعی کوچک بازگردد و CPU را در انحصار بگیرد. وقتی یک وظیفه خاتمه می‌یابد، از درخت حذف و منابعش آزاد می‌شوند. این عملیات باید با توازن بار و با تغییرات هم‌زمان روی CPUهای دیگر همگام شوند.

۳٫۱۲ توازن بار و وابستگی CPU

در یک سیستم SMP، CFS یک صف اجرای جداگانه برای هر CPU نگه می‌دارد. توازن بار می‌کوشد بار را در CPUها تقریباً برابر نگه دارد، در حالی که **وابستگی CPU** (مجموعه CPUهایی که یک وظیفه اجازه اجرا روی آن‌ها را دارد) و **محلیت حافظه نهان** را رعایت می‌کند. مهاجرت یک وظیفه به CPU دیگر می‌تواند بهره‌وری را بهبود دهد اما ممکن است خطاهای حافظه نهان و TLB را به همراه داشته باشد. لینوکس CPUها را در **دامنه‌های زمان‌بندی** سازمان می‌دهد که حافظه‌های نهان مشترک و توپولوژی NUMA را منعکس می‌کنند و متوازن‌کننده بار می‌کوشد وظایف را در داخل یک دامنه پیش از عبور از دامنه‌ها جابه‌جا کند. وابستگی از طریق `taskset` و `sched_setaffinity(2)` در فضای کاربر در معرض دید قرار می‌گیرد. توازن بار در فصل ۷ به‌تفصیل بحث می‌شود.

۳٫۱۳ زمان‌بندی گروهی و انصاف سلسله‌مراتبی

زمان‌بندی گروهی انصاف را از وظایف منفرد به گروه‌هایی از وظایف، که به‌طور معمول با گروه‌های کنترل (cgroups) تعریف می‌شوند، گسترش می‌دهد. هر گروه با یک موجودیت زمان‌بندی با وزن و زمان اجرای مجازی خود نمایش داده می‌شود و فرزندان گروه درون تخصیص گروه زمان‌بندی می‌شوند. این امر اشتراک منصفانه سلسله‌مراتبی را ممکن می‌سازد: یک گروه با وزن W_G سهمی از زمان CPU متناسب با W_G نسبت به سایر گروه‌ها دریافت می‌کند و درون گروه، وظایف با توجه به وزن‌های خود تخصیص گروه را به اشتراک می‌گذارند. زمان‌بندی گروهی به‌ویژه در محیط‌های کانتینری مهم است، جایی که ممکن است چندین کانتینر نیاز به ایزوله شدن از یکدیگر داشته باشند. کنترل‌کننده CPU در cgroup v2 محدودیت‌های پهنای باند را از طریق `cpu.max` و وزن را از طریق `cpu.weight` در معرض دید قرار می‌دهد و کنترل پهنای باند CFS می‌تواند گروهی را که از سهمیه‌اش فراتر می‌رود محدود کند.

۳٫۱۴ CFS و کلاس‌های زمان‌بندی لینوکس

CFS تنها زمان‌بند لینوکس نیست؛ بلکه پیاده‌سازی **کلاس زمان‌بندی منصفانه** است. سلسله‌مراتب کلاس‌های زمان‌بندی ترتیبی را تعیین می‌کند که سیاست‌ها مشورت می‌شوند. کلاس توقف بالاترین اولویت را دارد و به‌صورت درونی برای مهاجرت CPU و hotplug استفاده می‌شود. کلاس مهلت `SCHED_DEADLINE` را بر پایه EDF با کنترل پذیرش پیاده‌سازی می‌کند. کلاس بی‌درنگ `SCHED_FIFO` و `SCHED_RR` را با اولویت‌های ۱ تا ۹۹ پیاده می‌کند. کلاس منصفانه `SCHED_NORMAL`، `SCHED_BATCH` و `SCHED_IDLE` را پیاده می‌کند. کلاس بیکار رشته بیکار هر CPU را زمانی که هیچ وظیفه دیگری آماده اجرا نیست اجرا می‌کند. این سلسله‌مراتب بدین معناست که یک وظیفه بی‌درنگ همیشه بر وظیفه CFS پیش‌دستی خواهد کرد و یک وظیفه مهلت اگر مهلتش ایجاب کند بر وظیفه بی‌درنگ پیش‌دستی خواهد کرد. بنابراین CFS در یک ناحیه کران‌دار از فضای سیاست زمان‌بندی عمل می‌کند؛ نمی‌تواند تضمین‌های کلاس‌های بالاتر را نادیده بگیرد.

۳٫۱۵ CFS کلاسیک در برابر EEVDF

CFS کلاسیک موجودیتی را با کوچک‌ترین زمان اجرای مجازی انتخاب می‌کند. EEVDF، که در لینوکس ۶٫۶ به‌عنوان الگوریتم کلاس منصفانه معرفی شد، ایده زمان اجرای مجازی را حفظ می‌کند اما دو مفهوم را می‌افزاید: **تأخیر** و **مهلت مجازی**. تأخیر می‌سنجد که آیا یک موجودیت بیش از سهم منصفانه‌اش سرویس دریافت کرده یا کمتر: تأخیر مثبت به معنای آن است که موجودیت مستحق زمان CPU است و تأخیر منفی به معنای آن است که از سهمش فراتر رفته. EEVDF تنها موجودیت‌های با تأخیر نامنفی (موجودیت‌های واجد شرایط) را در نظر می‌گیرد و از میان آن‌ها آن را با نزدیک‌ترین مهلت مجازی که از برش زمانی درخواستی موجودیت مشتق می‌شود انتخاب می‌کند. این امر به وظایف

حساس به تأخیر اجازه می‌دهد برش‌های کوتاه‌تر درخواست کنند و مهلت‌های زودتری دریافت کنند و پاسخ‌دهی را بدون رها کردن انصاف بهبود می‌بخشد. EEVDF همچنین چندین اکتشافی و پارامتر قابل تنظیم CFS را با قواعد الگوریتمی جایگزین می‌کند و مکانیزم تأخیر پوسیده را برای وظایف خواب‌رفته معرفی می‌کند تا از سوءاستفاده آن‌ها با خواب کوتاه و بازنشانی تأخیر جلوگیری کند. گذار تدریجی است: لینوکس ۶٫۶ EEVDF را به‌عنوان پیاده‌سازی کلاس منصفانه معرفی کرد و هسته‌های بعدی به پالایش آن ادامه دادند. جدول ۳٫۳ تفاوت‌ها را خلاصه می‌کند.

جدول ۳٫۳ — CFS کلاسیک در برابر مفاهیم جدیدتر زمان‌بندی مرتبط با EEVDF.

جنبه	CFS کلاسیک	EEVDF (لینوکس ۶٫۶+)
کلید اصلی انتخاب	کوچک‌ترین `vruntime`	نزدیک‌ترین مهلت مجازی میان موجودیت‌های واجد شرایط با تأخیر نامنفی
معیار انصاف	زمان اجرای مجازی	تأخیر (سرویس مستحق یا فراتر رفته)
کنترل تأخیر	اکتشافی‌ها و گرانولاریتی بیدار شدن	برش درخواستی و مهلت مجازی
پارامترهای قابل تنظیم	چندین sysctl و اکتشافی	کمتر؛ قواعد الگوریتمی جای اکتشافی‌ها را می‌گیرند
مدیریت وظایف خواب‌رفته	جای‌گذاری نسبت به `min_vruntime`	تأخیر پوسیده در طول زمان اجرای مجازی
ساختار داده	درخت قرمز-سیاه بر اساس `vruntime`	درخت قرمز-سیاه با کلیدهای تأخیر و مهلت
نسخه هسته	۲٫۶٫۲۳ تا ۶٫۵ (به‌عنوان کلاس منصفانه)	۶٫۶ و بعد (کلاس منصفانه)

یادداشت نسخه

گزاره‌های این مقاله که «CFS» را توصیف می‌کنند، به الگوریتم کلاسیک کلاس منصفانه اشاره دارند مگر این‌که صراحتاً خلاف آن ذکر شده باشد. لینوکس ۶٫۶ منطق انتخاب CFS را با EEVDF در کلاس منصفانه جایگزین کرد. مبانی مفهومی — وزن‌ها، زمان اجرای مجازی، انصاف نسبی، زمان‌بندی گروهی و توازن بار — همچنان مرتبط هستند، اما جزئیات پیاده‌سازی و رفتار انتخاب متفاوت است.

۴. جریان کاری الگوریتم CFS

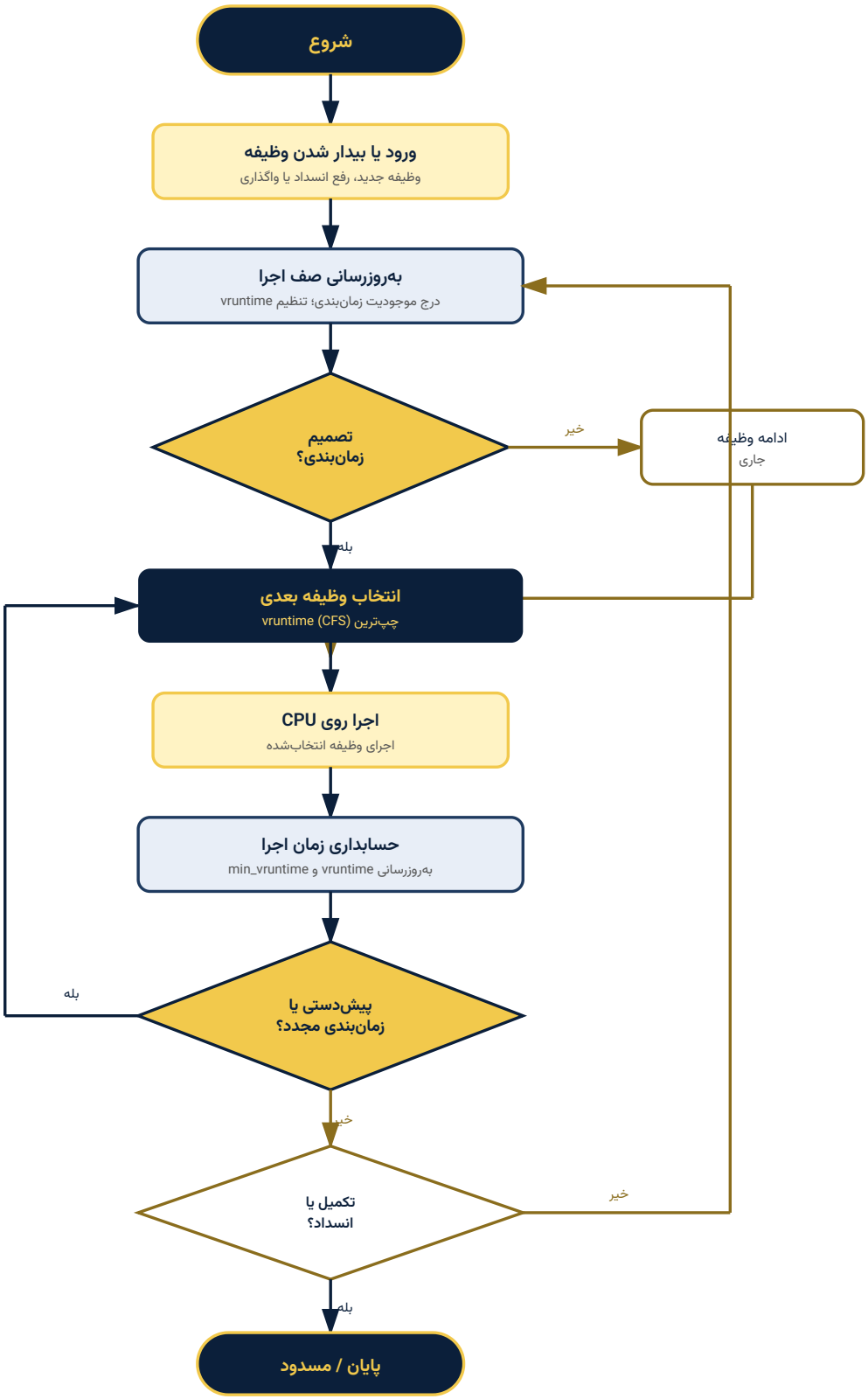
۴٫۱ گام‌به‌گام جریان کاری

توالی زیر جریان کاری مفهومی CFS کلاسیک را توصیف می‌کند. این توالی عامدانه عمومی است؛ یک نسخه هسته مشخص ممکن است گام‌هایی را بیفزاید، حذف یا بازآرایی کند.

1. **وظیفه آماده اجرا می‌شود.** یک وظیفه ایجاد، از خواب بیدار یا از انسداد خارج می‌شود. موجودیت زمان‌بندی آن با وزن و زمان اجرای مجازی‌اش مقداردهی یا به‌روزرسانی می‌شود.
2. **پارامترهای زمان‌بندی در نظر گرفته می‌شوند.** وزن موجودیت از مقدار nice آن مشتق و سلسله‌مراتب گروهی در صورت وجود لحاظ می‌شود.
3. **وظیفه وارد صف اجرای مربوطه می‌شود.** موجودیت در درخت قرمز-سیاه هر CPU کلیدخورده بر اساس `vruntime` درج می‌شود. موجودیت‌های جدید یا بیدارشده نسبت به `min_vruntime` جای‌گذاری می‌شوند تا مزیت ناعادلانه‌ای به دست نیاورند.
4. **زمان‌بند موجودیت‌های آماده اجرا را مقایسه می‌کند.** درخت موجودیت‌ها را بر اساس زمان اجرای مجازی مرتب می‌کند، بنابراین چپ‌ترین موجودیت همان با کوچک‌ترین زمان اجرای مجازی است.
5. **یک وظیفه انتخاب می‌شود.** در CFS کلاسیک چپ‌ترین موجودیت انتخاب می‌شود. در EEVDF موجودیت واجد شرایط با نزدیک‌ترین مهلت مجازی انتخاب می‌شود.
6. **وظیفه انتخاب‌شده اجرا می‌شود.** CPU به وظیفه انتخاب‌شده سوئیچ می‌کند و وضعیت ثبات و فضای آدرس آن را در صورت لزوم بازیابی می‌کند.
7. **زمان اجرا و زمان اجرای مجازی به‌روزرسانی می‌شوند.** زمان اجرای واقعی موجودیت حساب و زمان اجرای مجازی آن مطابق وزنش پیش می‌رود.
8. **یک تصمیم زمان‌بندی در صورت لزوم رخ می‌دهد.** وظیفه در حال اجرا ممکن است مسدود شود، واگذار کند، توسط کلاس با اولویت بالاتر پیش‌دستی شود یا به این دلیل پیش‌دستی شود که موجودیت دیگری از کلاس منصفانه زمان اجرای مجازی به‌اندازه کافی کوچک‌تری دارد.
9. **زمان‌بند وظیفه دیگری را انتخاب می‌کند یا اجرا را ادامه می‌دهد.** اگر وظیفه جاری همچنان بهترین انتخاب باشد ادامه می‌یابد؛ در غیر این صورت تعویض زمینه رخ می‌دهد.
10. **توازن بار ممکن است کار را بازتوزیع کند.** به‌صورت دوره‌ای یا بر اساس رویدادها، زمان‌بند ممکن است موجودیت‌های آماده اجرا را میان CPUها مهاجرت دهد تا بار را متوازن کند، مشروط به وابستگی و ملاحظات حافظه نهان.

۴٫۲ نمودار جریان لازم

شکل ۴٫۱ جریان کاری را به‌صورت یک نمودار جریان ارائه می‌دهد. نمودار نقاط تصمیم (لوزی) را از گام‌های فرایند (مستطیل) تفکیک می‌کند و حلقه بازگشت به انتخاب وظیفه را پس از اجرا و حسابداری نشان می‌دهد.



شکل ۴٫۱ — نمودار جریان جریان کاری زمان‌بندی CFS. حلقه از «حسابداری زمان اجرا» به «انتخاب وظیفه بعدی» بازمی‌گردد و عملکرد پیوسته زمان‌بند را نشان می‌دهد. مسیر سمت راست نشان می‌دهد که وظیفه‌ای که نه مسدود می‌شود و نه کامل می‌گردد به صف آماده اجرا بازمی‌گردد و ممکن است دوباره انتخاب شود. این نمودار مفهومی است و جزئیات وابسته به نسخه هسته مانند محاسبات تأخیر و مهلت مجازی EEVDF را نادیده می‌گیرد.

۴٫۳ مدل مفهومی در برابر پیاده‌سازی

جریان کاری بالا یک مدل آموزشی است. یک هسته عملی جزئیات اضافی بسیاری را پیاده می‌کند: ردگیری بار هر موجودیت (PELT)، زمان‌بندی گروهی سلسله‌مراتبی، توازن بار آگاه از NUMA و محدودسازی بی‌درنگ. قاعده انتخاب در CFS کلاسیک «چپ‌ترین `vruntime`» است، اما درخت ممکن است موجودیت‌های گروهی را در خود داشته باشد که فرزندانشان به‌صورت بازگشتی زمان‌بندی می‌شوند. قاعده حسابداری «`vruntime` به‌طور معکوس با وزن افزایش می‌یابد» است، اما هسته ممکن است `vruntime` را در زمان بیدار شدن، مهاجرت و تغییرات گروهی تنظیم کند. در EEVDF قاعده انتخاب «نزدیک‌ترین مهلت مجازی میان موجودیت‌های واجد شرایط با تأخیر نامنفی» است که معیاری الگوریتمی متفاوت است، هرچند هدف اشتراک نسبی یکسان را دنبال می‌کند. بنابراین خوانندگان باید جریان کاری را به‌عنوان یک داربست مفهومی در نظر بگیرند و برای نسخه مشخصی که مطالعه می‌کنند به مستندات هسته مراجعه کنند.

۵. شبه‌کد و مثال حل‌شده CFS

۵٫۱ شبه‌کد مفهومی

شبه‌کد زیر یک مدل آموزشی مستقل از زبان است. این یک کپی از پیاده‌سازی هسته لینوکس نیست و همگام‌سازی، زمان‌بندی گروهی، توازن بار و بسیاری از موارد حاشیه‌ای را نادیده می‌گیرد. هدف آن صریح ساختن الگوریتم مفهومی است.

```
// مدل آموزشی مفهومی CFS — کد منبع هسته نیست
// هر موجودیت زمان‌بندی دارد:
//   id، weight (از nice)، vruntime (زمان اجرای مجازی)، state

CONSTANT W0 = 1024          // وزن مرجع در nice صفر
CONSTANT MIN_GRANULARITY = 3_000_000  // نانوثانیه گویا
CONSTANT TARGET_LATENCY = 6_000_000   // نانوثانیه گویا

STRUCT Entity:
    id
    weight
    vruntime
    state        // RUNNABLE, RUNNING, BLOCKED, TERMINATED
    last_start

FUNCTION weight_from_nice(nice):
    // جدول جستجو؛ nice صفر <- 1024، هر پله ~1.25x
    RETURN lookup_weight(nice)

FUNCTION init_entity(id, nice):
    e = new Entity
    e.id = id
    e.weight = weight_from_nice(nice)
    e.vruntime = 0
    e.state = RUNNABLE
    RETURN e

FUNCTION enqueue(runqueue, e):
    // درج در درخت قرمز-سیاه کلیدخورده بر اساس vruntime
    // موجودیت‌های جدید یا بیدارشده نسبت به min_vruntime جای‌گذاری می‌شوند
    IF e.is_new_or_woken:
        e.vruntime = max(e.vruntime, runqueue.min_vruntime)
    tree_insert(runqueue.rb_tree, e)

FUNCTION pick_next(runqueue):
    IF runqueue.is_empty():
        RETURN idle_task
    // CFS کلاسیک: چپ‌ترین گره کوچک‌ترین vruntime را دارد
    e = tree_minimum(runqueue.rb_tree)
    RETURN e

FUNCTION update_vruntime(e, delta_exec):
    // افزایش وزن‌دار زمان اجرای مجازی
    e.vruntime = e.vruntime + delta_exec * (W0 / e.weight)

FUNCTION schedule(runqueue):
    current = runqueue.current
    next = pick_next(runqueue)

    IF next == current:
        RETURN  // ادامه اجرا

    IF current.state == RUNNING:
```

```
        current.state = RUNNABLE
        update_vruntime(current, now() - current.last_start)
        enqueue(runqueue, current)

    next.state = RUNNING
    next.last_start = now()
    context_switch(current, next)
    runqueue.current = next

FUNCTION on_tick(runqueue):
    current = runqueue.current
    delta = now() - current.last_start
    update_vruntime(current, delta)

    // بررسی این‌که آیا موجودیت دیگری باید اجرا شود
    IF should_preempt(runqueue, current):
        schedule(runqueue)

FUNCTION should_preempt(runqueue, current):
    candidate = pick_next(runqueue)
    IF candidate == current:
        RETURN false
    // گرانولاریتی بیدار شدن از پیش‌دستی بیش‌ازحد جلوگیری می‌کند
    RETURN (current.vruntime - candidate.vruntime) > wakeup_granularity()

FUNCTION on_block(e):
    e.state = BLOCKED
    tree_remove(e.runqueue.rb_tree, e)

FUNCTION on_wake(e, runqueue):
    e.state = RUNNABLE
    e.vruntime = max(e.vruntime, runqueue.min_vruntime)
    enqueue(runqueue, e)
    // پیش‌دستی را فعال کند، e ممکن است در صورت کافی بودن مزیت

FUNCTION on_terminate(e):
    e.state = TERMINATED
    tree_remove(e.runqueue.rb_tree, e)

// یادداشت ساده‌شده چند-CPU:
// هر CPU صف اجرای خود را دارد. توازن بار به‌طور دوره‌ای
// موجودیت‌های آماده اجرا را میان صف‌ها مهاجرت می‌دهد تا بار را برابر کند،
// مشروط به وابستگی و محدودیت‌های حافظه نهان/NUMA.
```

فهرست ۵٫۱ — شبه‌کد مفهومی برای CFS کلاسیک. ثابت‌ها و ساده‌سازی‌های گویا به کار رفته‌اند؛ هسته‌های عملی تفاوت‌های قابل‌توجهی دارند.

۵٫۲ مثال حل‌شده با سه وظیفه

سه وظیفه CPU-محور A، B و C را در نظر بگیرید که در زمان صفر آماده اجرا می‌شوند و در تمام بازه مورد مطالعه آماده اجرا باقی می‌مانند. مقادیر nice آن‌ها به‌ترتیب ۰، ۵ و ۱۰ است. با استفاده از وزن‌های گویای جدول ۳٫۲ داریم w_A = ۱۰۲۴، w_B = ۳۳۵ و w_C = ۱۱۰. وزن کل ۱۴۶۹ است. تحت رقابت پایدار، سهم‌های نسبی ایده‌آل عبارت‌اند از:

$$share_A = \frac{1024}{1469} \approx 69.7\% \tag{۵٫۱}$$

$$share_B = \frac{335}{1469} \approx 22.8\% \tag{۵٫۲}$$

$$share_C = \frac{110}{1469} \approx 7.5\% \tag{۵٫۳}$$

یک تأخیر زمان‌بندی ساده‌شده ۱۲ واحد زمانی فرض کنید. زمان‌بند یک برش زمانی هدف متناسب با وزن، مشروط به یک گرانولاریتی حداقل، اختصاص می‌دهد. برای گویایی، فرض کنید دوره زمان‌بندی نخست به برش‌های ۷، ۳ و ۲ واحد زمانی برای A، B و C تقسیم می‌شود. (محاسبه برش در یک هسته واقعی به `sched_latency_ns`، `sched_min_granularity_ns` و تعداد موجودیت‌های آماده اجرا بستگی دارد؛ اعداد این‌جا برای شفافیت حسابی انتخاب شده‌اند.) جدول ۵٫۱ پارامترهای اولیه را نشان می‌دهد.

جدول ۵٫۱ — پارامترهای اولیه وظایف برای مثال حل‌شده.

وظیفه	nice	وزن	سهم ایده‌آل	برش نخست
A	۰	۱۰۲۴	۶۹٫۷٪	۷
B	۵	۳۳۵	۲۲٫۸٪	۳
C	۱۰	۱۱۰	۷٫۵٪	۲

ما زمان اجرای مجازی را با معادله ۳٫۲ و $W_0 = 1024$ ردگیری می‌کنیم. برای وظیفه A، $\Delta vruntime = \Delta exec \times 1024/1024 = \Delta exec$. برای وظیفه B، $\Delta vruntime = \Delta exec \times 1024/335 \approx 3{,}06 \times \Delta exec$. برای وظیفه C، $\Delta vruntime = \Delta exec \times 1024/110 \approx 9{,}31 \times \Delta exec$. زمان‌بند همیشه موجودیت با کوچک‌ترین زمان اجرای مجازی را انتخاب می‌کند. جدول ۵٫۲ تحول را در چند بازه ردگیری می‌کند.

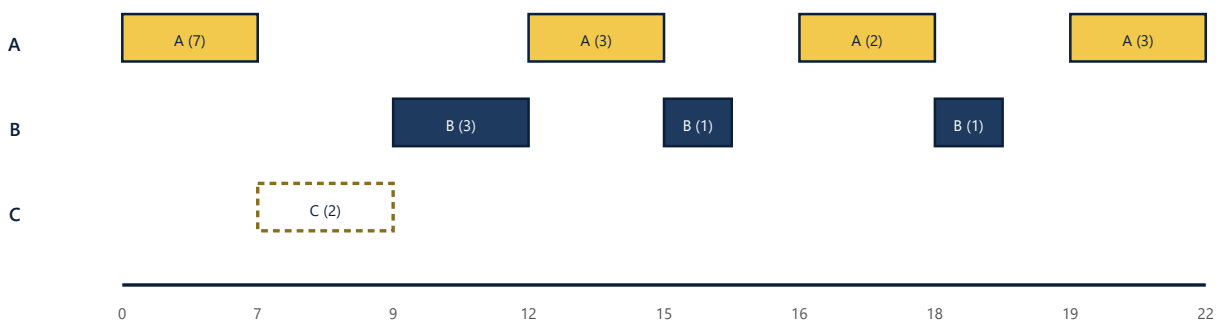
جدول ۵٫۲ — تحول زمان اجرای مجازی در مثال حل‌شده (مقادیر گویا).

گام	رویداد	vruntime A	vruntime B	vruntime C	انتخاب بعدی
۰	همه در t=۰ آماده اجرا	۰	۰	۰	A (تساوی؛ کمترین شناسه)
۱	A برای ۷ واحد اجرا	۷٫۰۰	۰	۰	C
۲	C برای ۲ واحد اجرا	۷٫۰۰	۰	۱۸٫۶۲	B
۳	B برای ۳ واحد اجرا	۷٫۰۰	۹٫۱۸	۱۸٫۶۲	A
۴	A برای ۳ واحد اجرا	۱۰٫۰۰	۹٫۱۸	۱۸٫۶۲	B
۵	B برای ۱ واحد اجرا	۱۰٫۰۰	۱۲٫۲۴	۱۸٫۶۲	A
۶	A برای ۲ واحد اجرا	۱۲٫۰۰	۱۲٫۲۴	۱۸٫۶۲	B
۷	B برای ۱ واحد اجرا	۱۲٫۰۰	۱۵٫۳۰	۱۸٫۶۲	A
۸	A برای ۳ واحد اجرا	۱۵٫۰۰	۱۵٫۳۰	۱۸٫۶۲	B

دو مشاهده حاصل می‌شود. نخست، وظیفه با کوچک‌ترین زمان اجرای مجازی همیشه انتخاب می‌شود؛ این معنای عملیاتی «کاملاً منصفانه» در CFS کلاسیک است. دوم، وظیفه A با وزن بالاتر زمان اجرای مجازی را کندتر انباشته می‌کند و بنابراین در بلندمدت زمان CPU بیشتری دریافت می‌کند. مثال عمداً ساده‌شده است: گرانولاریتی بیدار شدن، زمان‌بندی گروهی و این واقعیت که وظایف واقعی مسدود و مهاجرت می‌کنند را نادیده می‌گیرد. همچنین فرض می‌کند هر سه وظیفه آماده اجرا باقی می‌مانند، که رقابت را بیشینه می‌کند.

۵٫۳ تخصیص انتظاری در برابر مشاهده‌شده

در یک بازه به‌اندازه کافی طولانی با آماده اجرا بودن مداوم همه وظایف، تخصیص CPU مشاهده‌شده باید به نسبت‌های ایده‌آل در معادلات ۵٫۱ تا ۵٫۳ نزدیک شود. در ردگیری کوتاه بالا، تخصیص ناهموار است زیرا زمان‌بند میان وظایف جابه‌جا می‌شود تا زمان اجرای مجازی را برابر کند و ردگیری پیش از همگرایی پایان می‌یابد. این انتظار می‌رود: انصاف نسبی یک ویژگی بلندمدت است، نه تضمینی برای هر پنجره کوتاه. شکل ۵٫۱ یک نمودار گانت برای چند بازه نخست نشان می‌دهد.



شکل ۵٫۱ — نمودار گانت برای مثال حل‌شده سه‌وظیفه‌ای. محور عمودی وظایف را مشخص می‌کند؛ محور افقی زمان است. زمان‌بندی گویا است و به مقیاس رسم نشده. CFS واقعی بازه‌ها را ریزتر درهم می‌آمیزد و از طول برش‌های متفاوت بر اساس پارامترهای قابل تنظیم هسته استفاده می‌کند.

یادداشت مفروضات

همه مقادیر عددی این فصل آموزشی هستند. آن‌ها اندازه‌گیری از یک نسخه هسته، CPU یا بار کاری خاص نیستند. رفتار واقعی CFS به `sched_latency_ns`، `sched_min_granularity_ns`، `sched_wakeup_granularity_ns`، تعداد وظایف آماده اجرا، توپولوژی CPU و پیکربندی cgroup بستگی دارد. خوانندگان نباید جدول‌ها یا نمودار گانت را به‌عنوان نتایج محک تفسیر کنند.

۶. ویژگی‌ها و مزایای اصلی CFS

۶٫۱ اشتراک نسبی CPU

CFS یک مکانیزم اصولی برای اشتراک نسبی CPU از طریق وزن‌های مشتق از مقادیر nice فراهم می‌کند. برخلاف زمان‌بندهایی که از برش‌های زمانی ثابت و تنظیمات اولویت ad hoc استفاده می‌کنند، CFS زمان اجرا را با وزن نرمال می‌کند و از یک ساعت مجازی واحد برای مقایسه موجودیت‌ها بهره می‌برد. این امر هدف انصاف را صریح و قابل حسابرسی می‌سازد. اهمیت عملی این است که مدیران می‌توانند درباره سهم‌های نسبی CPU با نسبت‌های وزن استدلال کنند و برنامه‌ها می‌توانند از طریق رابط استاندارد nice اولویت پایین‌تر یا بالاتر درخواست کنند. محدودیت این است که وزن‌ها نسبی هستند، نه رزرو: اگر یک وظیفه با وزن بالا مسدود شود، سهمش میان وظایف آماده اجرا بازتوزیع می‌شود و تضمینی وجود ندارد که یک وظیفه مقدار مطلقی از زمان CPU دریافت کند.

۶٫۲ انصاف میان وظایف آماده اجرا

ویژگی تعریف‌کننده CFS انصاف میان وظایف آماده اجراست. از آن‌جا که زمان‌بند همیشه موجودیت با کوچک‌ترین زمان اجرای مجازی را انتخاب می‌کند، هیچ وظیفه آماده اجرا نمی‌تواند به‌طور نامحدود نادیده گرفته شود تا زمانی که آماده اجرا بماند و زمان‌بند به‌طور عادی عمل کند. این خطر گرسنگی پایدار CPU را میان وظایف کلاس منصفانه کاهش می‌دهد، هرچند گرسنگی ناشی از کلاس‌های زمان‌بندی با اولویت بالاتر، محدودسازی cgroup یا بن‌بست منابع را از بین نمی‌برد (فصل ۱۱). انصاف بدون نیاز به طبقه‌بندی تعاملی/CPU-محور به دست می‌آید که CFS را نسبت به زمان‌بندهای اکتشافی در برابر تغییرات بار کاری مقاوم‌تر می‌کند.

۶٫۳ پشتیبانی از اولویت‌های مختلف وظیفه

CFS از اولویت‌های نسبی از طریق مقادیر nice و در زمان‌بندی گروهی از طریق وزن‌های گروه کنترل پشتیبانی می‌کند. این امکان به یک سرور اجازه می‌دهد به یک فرایند پایگاه داده سهم بیشتری نسبت به یک کار پشتیبان‌گیری بدهد، یا به یک رومیزی اجازه می‌دهد به یک رشته صوتی مقدار nice کوچک‌تری نسبت به ایندکسر فایل بدهد. مکانیزم اولویت نرم است: هر پله nice وزن را تقریباً ۱٫۲۵ برابر تغییر می‌دهد، بنابراین کاربران می‌توانند سهم‌ها را دقیق تنظیم کنند. اما مقادیر nice اولویت سخت فراهم نمی‌کنند: یک وظیفه با nice -۲۰ همچنان می‌تواند توسط یک وظیفه بی‌درنگ پیش‌دستی شود و یک وظیفه با nice پایین همچنان تحت رقابت مقداری زمان CPU دریافت می‌کند. این عامدانه است؛ CFS یک زمان‌بند منصف است، نه یک زمان‌بند اولویت سخت.

۶٫۴ پاسخ‌دهی تحت بارهای کاری ترکیبی

حسابداری زمان اجرای مجازی CFS به‌طور طبیعی وظایف تعاملی را جای می‌دهد. وظیفه‌ای که خوابیده است زمان اجرای مجازی نسبتاً کوچکی در هنگام بیدار شدن دارد (با تبعیت از قواعد جای‌گذاری)، بنابراین می‌تواند بدون طبقه‌بندی صریح به‌عنوان تعاملی به‌موقع زمان‌بندی شود. گرانولاریتی بیدار شدن از پیش‌دستی بیش‌ازحد توسط وظایف تازه بیدارشده جلوگیری می‌کند و پاسخ‌دهی را با سربار تعویض زمینه متوازن می‌سازد. در عمل، CFS پاسخ‌دهی رومیزی را

نسبت به زمان‌بند O(1) بهبود داد، هرچند بارهای کاری حساس به تأخیر همچنان می‌توانند تحت رقابت سنگین CPU آسیب ببینند. EEVDF کنترل تأخیر را بیشتر با اجازه دادن به وظایف برای درخواست برش‌های زمانی کوتاه‌تر و با استفاده از مهلت‌های مجازی بهبود می‌بخشد.

۶٫۵ مدیریت کارآمد وظایف آماده اجرا

درخت قرمز-سیاه درج و حذف را در O(log n) و جستجوی کمینه را در O(1) در CFS کلاسیک فراهم می‌کند. این برای اندازه‌های معمول صف اجرا کارآمد است و به‌خوبی تا هزاران موجودیت آماده اجرا مقیاس می‌یابد. درخت خودمتوازن است، بنابراین رفتار بدترین حالت کران‌دار است. صف اجرا همچنین `min_vruntime` و تجمیع‌های بار را نگه می‌دارد که جای‌گذاری و توازن بار را پشتیبانی می‌کنند. محدودیت این است که درخت باید با قفل‌ها روی هر CPU محافظت شود و رقابت می‌تواند در بارهای کاری حاوی بسیاری از وظایف کوتاه‌عمر یا مهاجرت‌های مکرر به گلوگاه تبدیل شود.

۶٫۶ ملاحظات مقیاس‌پذیری

CFS از صف‌های اجرای هر CPU استفاده می‌کند که رقابت قفل را نسبت به یک صف اجرای سراسری واحد کاهش می‌دهد. هر CPU به‌طور مستقل زمان‌بندی می‌کند و توازن بار در CPUها عمل می‌کند. این طراحی تا سیستم‌های چند هسته‌ای بزرگ مقیاس می‌یابد اما مقیاس‌پذیری نامحدود نیست: ترافیک هم‌پیوستگی حافظه نهان، هزینه‌های مهاجرت و اثرات NUMA می‌توانند عملکرد را تخریب کنند. سلسله‌مراتب دامنه‌های زمان‌بندی و ردگیری بار هر موجودیت هسته برای مدیریت این سازش‌ها طراحی شده‌اند. درس عملی این است که مقیاس‌پذیری CFS به توپولوژی، بار کاری و پیکربندی بستگی دارد، نه فقط به ساختار داده.

۶٫۷ پشتیبانی SMP

CFS از چندپردازشی متقارن از طریق صف‌های اجرای هر CPU، دامنه‌های زمان‌بندی و توازن بار پشتیبانی می‌کند. هر CPU می‌تواند تصمیم‌های زمان‌بندی محلی را بدون مراجعه به یک قفل سراسری اتخاذ کند و متوازن‌کننده بار به‌طور دوره‌ای کار را به CPUهای بیکار منتقل می‌کند. وابستگی CPU به مدیران اجازه می‌دهد وظایف را به CPUهای مشخص برای محلیت حافظه نهان یا ایزولاسیون سنجاق کنند. این ویژگی‌ها CFS را برای سرورها، رومیزی‌ها و سیستم‌های نهفته چند هسته‌ای مناسب می‌سازد. سازش میان انصاف و سربار مهاجرت در فصل ۷ بحث می‌شود.

۶٫۸ توازن بار

توازن بار برای بهره‌گیری از CPUهای متعدد ضروری است. متوازن‌کننده بار CFS بار هر صف اجرا، توپولوژی دامنه‌های زمان‌بندی و وابستگی وظایف را در نظر می‌گیرد. می‌کوشد وظایف را از CPUهای پرمشغله به CPUهای بیکار منتقل کند در حالی که محلیت حافظه نهان و NUMA را رعایت می‌کند. متوازن‌کننده در هر لحظه کاملاً منصف نیست؛ به‌صورت دوره‌ای عمل می‌کند و از اکتشافی‌ها برای اجتناب از مهاجرت بیش‌ازحد استفاده می‌کند. این یک سازش عامدانه است: توازن لحظه‌ای کامل نیازمند مهاجرت مکرر است و می‌تواند محلیت حافظه نهان را از بین ببرد.

۶٫۹ وابستگی CPU

وابستگی CPU مجموعه CPUهایی را که یک وظیفه ممکن است روی آن‌ها اجرا شود محدود می‌کند. این برای ایزوله کردن وظایف حساس به تأخیر، بهبود محلیت حافظه نهان و پیاده‌سازی پارتیشن‌بندی CPU مفید است. CFS در طول توازن بار وابستگی را رعایت می‌کند، هرچند ممکن است همچنان وظیفه‌ای را مهاجرت دهد اگر ماسک وابستگی اجازه

دهد و عدم توازن بار قابل توجه باشد. وابستگی از طریق `taskset` و `sched_setaffinity(2)` در معرض دید قرار می‌گیرد. محدودیت این است که وابستگی بیش‌ازحد محدودکننده می‌تواند CPUها را بیکار بگذارد در حالی که دیگران اضافه‌بار دارند، بنابراین وابستگی باید با احتیاط استفاده شود.

۶٫۱۰ زمان‌بندی گروهی

زمان‌بندی گروهی انصاف را به سلسله‌مراتب‌هایی از وظایف گسترش می‌دهد که برای کانتینرها و سیستم‌های چند-مستأجری ضروری است. یک گروه کنترل می‌تواند وزنی یا محدودیت پهنای باند داشته باشد و CFS سهم گروه را اجرا می‌کند در حالی که فرزندانش را به‌طور منصفانه زمان‌بندی می‌کند. این درجه‌ای از ایزولاسیون میان بارهای کاری فراهم می‌کند و به مدیران اجازه می‌دهد درباره تخصیص منابع در سطح گروه استدلال کنند. پیچیدگی در این است که حسابداری سلسله‌مراتبی و محدودسازی به‌طور غیرساده‌ای تعامل می‌کنند و پیکربندی نادرست می‌تواند به محدودسازی یا انصاف غیرمنتظره منجر شود.

۶٫۱۱ یکپارچگی با کلاس‌های زمان‌بندی لینوکس

CFS در سلسله‌مراتب کلاس‌های زمان‌بندی لینوکس یکپارچه است که به آن اجازه می‌دهد با زمان‌بندی بی‌درنگ و مهلت هم‌زیستی کند. وظایف بی‌درنگ بر وظایف کلاس منصفانه پیش‌دستی می‌کنند و وظایف مهلت اولویت بالاتری از وظایف بی‌درنگ دارند. این طراحی لایه‌ای به یک هسته واحد اجازه می‌دهد هم بارهای کاری همه‌منظوره و هم بی‌درنگ را سرویس دهد، به شرطی که وظایف بی‌درنگ به‌درستی پیکربندی و محدود شوند. CFS خود نیازی ندارد تضمین‌های مهلت را پیاده‌سازی کند؛ این مسئولیت را به کلاس‌های مهلت و بی‌درنگ واگذار می‌کند.

۶٫۱۲ مزایا و محدودیت‌های عملی

جدول ۶٫۱ مزایا و معایب اصلی CFS را به‌عنوان یک زمان‌بند همه‌منظوره خلاصه می‌کند.

جدول ۶٫۱ — مزایا و معایب CFS.

مزایا	معایب / محدودیت‌ها
انصاف نسبی اصولی از طریق زمان اجرای مجازی و وزن‌ها	بدون تضمین مهلت بی‌درنگ سخت
عدم نیاز به طبقه‌بندی وظایف به تعاملی یا CPU-محور	انصاف کوتاه‌مدت می‌تواند از نسبت‌های بلندمدت منحرف شود
عملیات کارآمد $O(\log n)$ روی صف اجرا	رقابت قفل هر CPU تحت بارهای کاری شدید
مقیاس‌پذیری خوب از طریق صف‌های اجرای هر CPU	سربار مهاجرت و حافظه نهان/NUMA
پشتیبانی از مقادیر nice و زمان‌بندی گروهی	زمان‌بندی گروهی و کنترل پهنای باند پیچیدگی می‌افزاید
یکپارچگی با کلاس‌های بی‌درنگ و مهلت	رفتار به نسخه هسته و پیکربندی بستگی دارد
پاسخ‌دهی رومیزی بهبودیافته نسبت به زمان‌بندهای قدیمی‌تر	بارهای کاری حساس به تأخیر ممکن است همچنان به ویژگی‌های بی‌درنگ یا EEVDF نیاز داشته باشند

۷. زمان‌بندی چندپردازنده و چند هسته‌ای

۷٫۱ چندپردازشی متقارن (SMP)

در یک سیستم SMP، همه CPUها همان حافظه را به اشتراک می‌گذارند و توسط سیستم‌عامل به‌عنوان همتا در نظر گرفته می‌شوند. هر CPU می‌تواند هر وظیفه‌ای را اجرا کند، مشروط به محدودیت‌های وابستگی. زمان‌بند نه‌تنها باید تصمیم بگیرد کدام وظیفه اجرا شود بلکه روی کدام CPU. این نیازمند صف‌های اجرای هر CPU، همگام‌سازی میان CPUها و یک مکانیزم توازن بار است. SMP مدل غالب برای سرورها، رومیزی‌ها و دستگاه‌های همراه است، هرچند سیستم‌های ناهمگن (big.LITTLE) و NUMA پیچیدگی بیشتری می‌افزایند.

۷٫۲ هسته‌های متعدد CPU و رشته‌های سخت‌افزاری

پردازنده‌های مدرن هسته‌های فیزیکی متعدد و اغلب رشته‌های سخت‌افزاری چندرشته‌ای هم‌زمان (SMT) را در معرض دید قرار می‌دهند. رشته‌های سخت‌افزاری واحدهای اجرا و حافظه‌های نهان را به اشتراک می‌گذارند، بنابراین معادل هسته‌های مستقل نیستند. زمان‌بند باید توپولوژی را لحاظ کند: رشته‌های سخت‌افزاری خواهر و برادر، حافظه‌های نهان سطح آخر مشترک و گره‌های NUMA. دامنه‌های زمان‌بندی لینوکس این توپولوژی را کدگذاری می‌کنند و توازن بار می‌کوشد وظایف را به‌گونه‌ای جای‌گذاری کند که اشتراک حافظه نهان را رعایت کند و از ترافیک غیرضروری بین گره‌ها پرهیز نماید.

۷٫۳ صف‌های اجرای هر CPU

هر CPU دارای `cfs_rq` و درخت قرمز-سیاه خود از موجودیت‌های آماده اجراست. تصمیم‌های زمان‌بندی محلی را می‌توان بدون کسب یک قفل سراسری اتخاذ کرد که مقیاس‌پذیری را بهبود می‌بخشد. اما صف‌های هر CPU می‌توانند نامتوازن شوند: یک CPU ممکن است بسیاری از وظایف آماده اجرا داشته باشد در حالی که دیگری بیکار است. توازن بار مسئول تصحیح این عدم توازن است، مشروط به وابستگی و محدودیت‌های محلیت.

۷٫۴ توازن بار میان CPUها

توازن بار در سطوح متعدد عمل می‌کند. توازن دوره‌ای در یک بازه پیکربندی‌شده اجرا می‌شود و می‌کوشد بار را در CPUهای درون یک دامنه زمان‌بندی برابر کند. توازن رویدادمحور ممکن است وقتی یک CPU بیکار می‌شود، وقتی یک وظیفه بیدار می‌شود یا وقتی یک وظیفه وابستگی‌اش را تغییر می‌دهد رخ دهد. متوازن‌کننده بار هر صف اجرا، تعداد موجودیت‌های آماده اجرا و هزینه مهاجرت را در نظر می‌گیرد. ممکن است وظایفی را از CPUهای پرمشغله بیرون بکشد یا وظایفی را به CPUهای بیکار هل دهد. هدف نگه داشتن همه CPUها به‌طور معقول بهره‌ور بدون مهاجرت بیش‌ازحد است.

۷٫۵ مهاجرت وظیفه

مهاجرت وظیفه یک وظیفه آماده اجرا را از صف اجرای یک CPU به صف دیگری منتقل می‌کند. مهاجرت می‌تواند بهره‌وری را بهبود دهد و تأخیر را کاهش دهد، اما هزینه دارد: مجموعه کاری وظیفه ممکن است دیگر در حافظه نهان CPU مقصد نباشد، ورودی‌های TLB ممکن است باطل شوند و وظیفه ممکن است نیاز به بازکسب قفل‌ها داشته

باشد. CFS می‌کوشد این هزینه‌ها را در برابر مزایای توزیع بار متوازن کند. مهاجرت همچنین توسط وابستگی CPU و سلسله‌مراتب دامنه‌های زمان‌بندی محدود می‌شود.

۷٫۶ وابستگی CPU و محلیت حافظه نهان

وابستگی CPU مجموعه CPUهایی را که یک وظیفه ممکن است روی آن‌ها اجرا شود محدود می‌کند. از آن برای بهبود محلیت حافظه نهان (با نگه داشتن وظیفه روی CPUی که داده‌اش در حافظه نهان است)، برای ایزوله کردن وظایف حساس به تأخیر و برای پیاده‌سازی پارتیشن‌بندی CPU استفاده می‌شود. وابستگی می‌تواند نرم (یک نشانه) یا سخت (یک ماسک اجراشده توسط زمان‌بند) باشد. CFS ماسک‌های وابستگی سخت را رعایت می‌کند و ممکن است وظایف را درون مجموعه مجاز مهاجرت دهد. وابستگی بیش‌ازحد محدودکننده می‌تواند باعث عدم توازن بار شود، بنابراین باید با احتیاط استفاده شود.

۷٫۷ مفاهیم دامنه زمان‌بندی

یک دامنه زمان‌بندی گروهی از CPUهاست که یک ویژگی زمان‌بندی مانند حافظه نهان سطح آخر یا یک گره NUMA را به اشتراک می‌گذارند. دامنه‌ها سلسله‌مراتبی سازمان‌دهی می‌شوند: پایین‌ترین سطح ممکن است شامل خواهر و برادرهای SMT باشد، سطح بعدی ممکن است شامل هسته‌های مشترک یک حافظه نهان باشد و بالاترین سطح ممکن است گره‌های NUMA را در بر بگیرد. توازن بار می‌کوشد ابتدا در پایین‌ترین دامنه توازن برقرار کند و تنها در صورت لزوم به دامنه‌های بالاتر برود. این سلسله‌مراتب هزینه مهاجرت را با ترجیح CPUهای نزدیک کاهش می‌دهد.

۷٫۸ ملاحظات NUMA

در یک سیستم NUMA، تأخیر دسترسی به حافظه به موقعیت فیزیکی حافظه نسبت به CPU بستگی دارد. مهاجرت یک وظیفه به یک CPU روی یک گره NUMA دیگر می‌تواند بهره‌وری CPU را بهبود دهد اما ممکن است تأخیر حافظه را افزایش دهد اگر داده‌های وظیفه روی گره اصلی بمانند. توازن بار آگاه از NUMA در CFS هم بار CPU و هم محلیت حافظه را در نظر می‌گیرد. سازش پیچیده است و سیاست بهینه به الگوهای دسترسی حافظه بار کاری و توپولوژی اتصال سیستم بستگی دارد.

۷٫۹ انصاف در برابر سربار مهاجرت

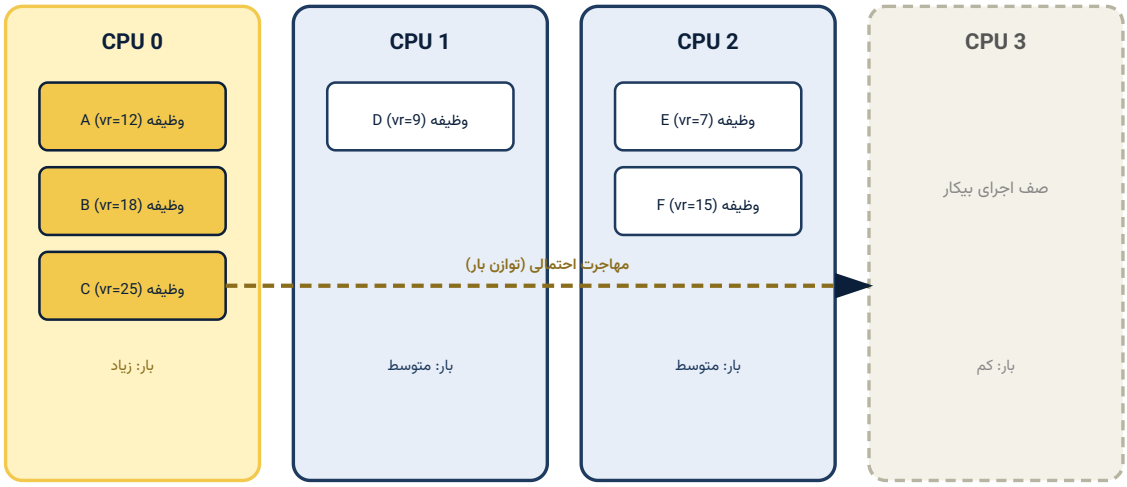
توازن بار کامل در هر لحظه نیازمند مهاجرت مکرر است که محلیت حافظه نهان را از بین می‌برد و سربار را افزایش می‌دهد. بنابراین CFS به‌صورت دوره‌ای متوازن می‌کند و از هیسترزیس برای اجتناب از جابه‌جایی غیرضروری وظایف استفاده می‌کند. این بدان معناست که عدم توازن کوتاه‌مدت به بهای سربار کمتر مهاجرت و رفتار بهتر حافظه نهان پذیرفته می‌شود. سازش تا حدی قابل تنظیم است اما سازش بنیادین ذاتی زمان‌بندی چند هسته‌ای است.

۷٫۱۰ همگام‌سازی و رقابت

صف‌های اجرای هر CPU در زمان مهاجرت وظایف، وقتی توازن بار وضعیت صف از راه دور را می‌خواند و وقتی پارامترهای زمان‌بندی تغییر می‌کنند نیاز به همگام‌سازی دارند. رقابت قفل می‌تواند در بارهای کاری حاوی بسیاری از وظایف کوتاه‌عمر یا مهاجرت‌های مکرر به گلوگاه تبدیل شود. هسته از قفل‌های هر CPU، به‌روزرسانی کپی-خواندنی (RCU) و ساختاردهی دقیق داده برای کاهش رقابت استفاده می‌کند. با این حال، بارهای کاری شدید می‌توانند محدودیت‌های مقیاس‌پذیری را آشکار کنند که یکی از انگیزه‌های پژوهش جاری در زمینه زمان‌بندی است.

۷٫۱۱ نمودار چندپردازنده

شکل ۷٫۱ چهار CPU را با صف اجرای هر یک و مهاجرت‌های ممکن وظیفه نشان می‌دهد. CPU 0 به‌شدت بارگذاری شده، CPU 3 بیکار است و متوازن‌کننده بار ممکن است وظیفه‌ای را از CPU 0 به CPU 3 مهاجرت دهد، مشروط به وابستگی و ملاحظات حافظه نهان.



هر CPU یک صف اجرای محلی دارد. متوازن‌کننده بار ممکن است موجودیت آماده اجرا را از یک CPU پرمشغله به یک CPU بیکار منتقل کند، مشروط به وابستگی و محدودیت‌های حافظه نهان/NUMA.

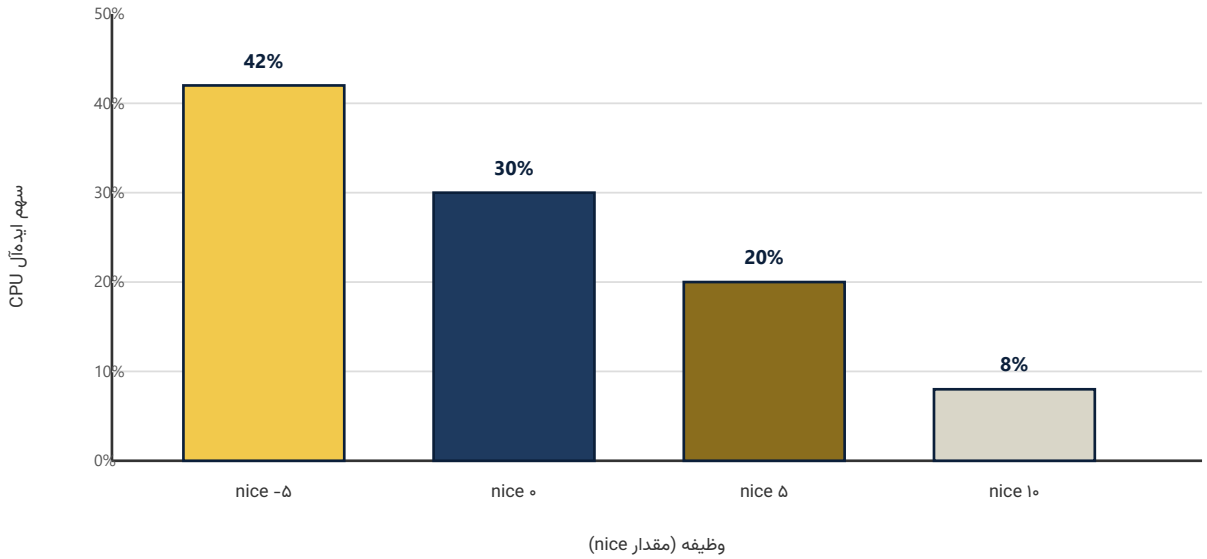
شکل ۷٫۱ — صف‌های اجرای هر CPU و مهاجرت احتمالی وظیفه در یک سیستم SMP. فلش خط‌چین نشان‌دهنده یک مهاجرت کاندید از CPU 0 به CPU 3 است. توازن بار واقعی دامنه‌های زمان‌بندی، وابستگی حافظه نهان و توپولوژی NUMA را در نظر می‌گیرد و لزوماً وظیفه نشان‌داده‌شده را مهاجرت نمی‌دهد.

۸. ارزیابی عملکرد و نمودارها

این فصل نمودارهای گویایی ارائه می‌دهد که ویژگی‌های کیفی و کمی CFS را توضیح می‌دهند. **همه مقادیر شبیه‌سازی‌شده یا آموزشی هستند، نه اندازه‌گیری‌های محک.** نمودارها برای کمک به خوانندگان در استدلال درباره رفتار زمان‌بندی طراحی شده‌اند؛ آن‌ها را نباید به‌عنوان شواهد درباره عملکرد یک هسته خاص روی یک بار کاری خاص ارجاع داد.

۸٫۱ تخصیص گویای CPU

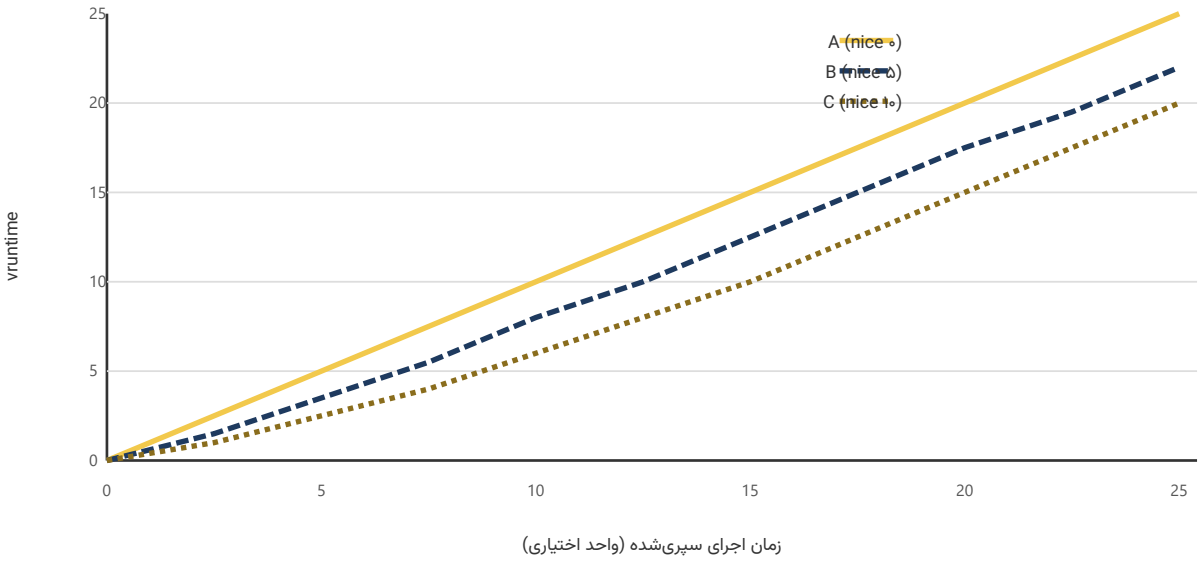
شکل ۸٫۱ سهم‌های ایده‌آل بلندمدت CPU را برای چهار وظیفه با مقادیر ۵ ،۰ ،۵- nice و ۱۰ مقایسه می‌کند، با فرض آماده اجرا بودن مداوم همه و نادیده گرفتن زمان‌بندی گروهی. میله‌ها نسبت‌های ایده‌آل محاسبه‌شده از جدول وزن را نشان می‌دهند، نه تخصیص اندازه‌گیری‌شده. هدف نشان دادن تأثیر مقادیر nice بر استحقاق است.



شکل ۸٫۱ — تخصیص ایده‌آل گویای CPU میان چهار وظیفه آماده اجرای مداوم با مقادیر nice متفاوت. مقادیر تقریب‌های آموزشی مشتق از نسبت‌های وزن هستند، نه اندازه‌گیری از یک سیستم در حال اجرا. تخصیص واقعی با بار کاری، نسخه هسته و پیکربندی cgroup تغییر می‌کند.

۸٫۲ تحول زمان اجرای مجازی

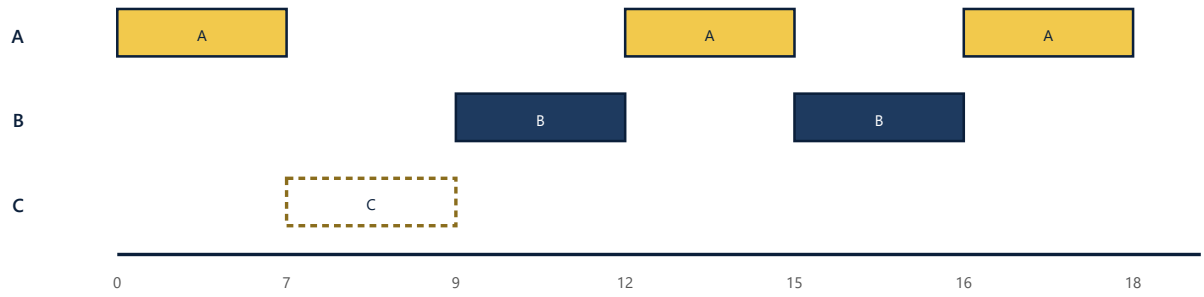
شکل ۸٫۲ زمان اجرای مجازی سه وظیفه را تحت مدل ساده‌شده فصل ۵ ردگیری می‌کند. وظیفه A مقدار nice صفر، وظیفه B مقدار nice ۵ و وظیفه C مقدار nice ۱۰ دارد. شیب‌ها متفاوت‌اند زیرا زمان اجرای مجازی برای وظایف با وزن بالاتر کندتر انباشته می‌شود. زمان‌بند در هر گام وظیفه با کوچک‌ترین زمان اجرای مجازی را انتخاب می‌کند و درهم‌آمیزی مشخصه را تولید می‌کند.



شکل ۸٫۲ — تحول گویای زمان اجرای مجازی برای سه وظیفه با وزن‌های متفاوت. وظایف با وزن بالاتر زمان اجرای مجازی را کندتر انباشته می‌کنند. شیب‌ها آموزشی هستند و نماینده یک ردگیری هسته خاص نیستند.

۸٫۳ نمودار گانت بازه‌های اجرا

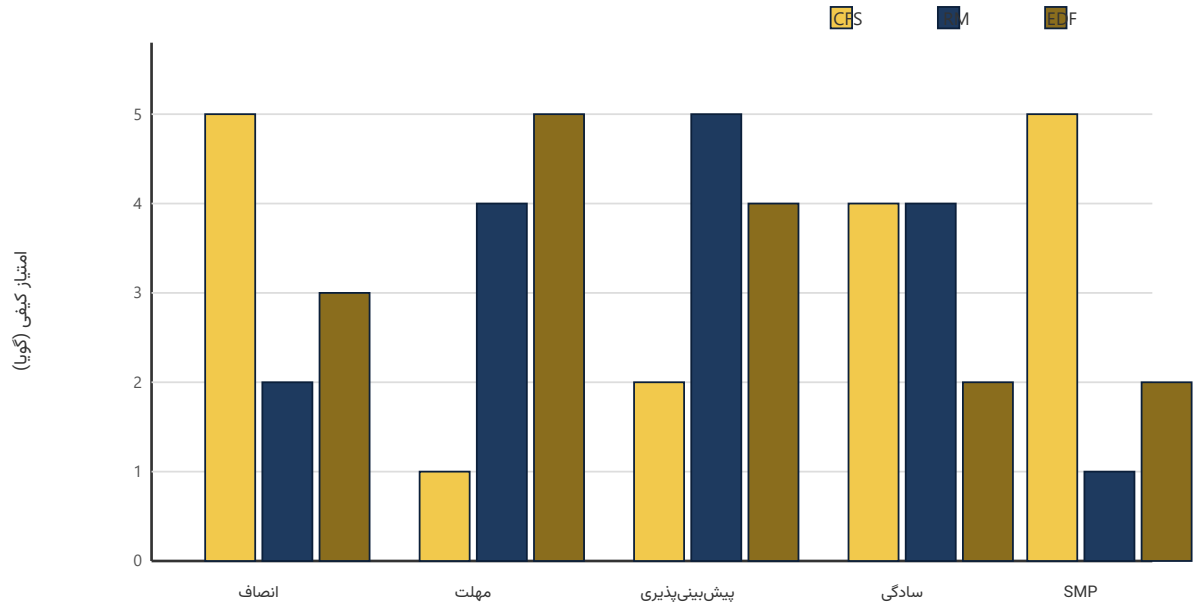
شکل ۸٫۳ یک نمودار گانت فشرده برای مثال حل‌شده ارائه می‌دهد و تأکید می‌کند که زمان‌بند میان وظایف جابه‌جا می‌شود تا زمان‌های اجرای مجازی‌شان را متوازن نگه دارد.



شکل ۸٫۳ — نمودار گانت بازه‌های اجرای گویای CFS. نمودار شماتیک است؛ برش‌های CFS واقعی کوتاه‌ترند و به تعداد موجودیت‌های آماده اجرا و پارامترهای هسته بستگی دارند.

۸٫۴ مقایسه کیفی CFS، RM و EDF

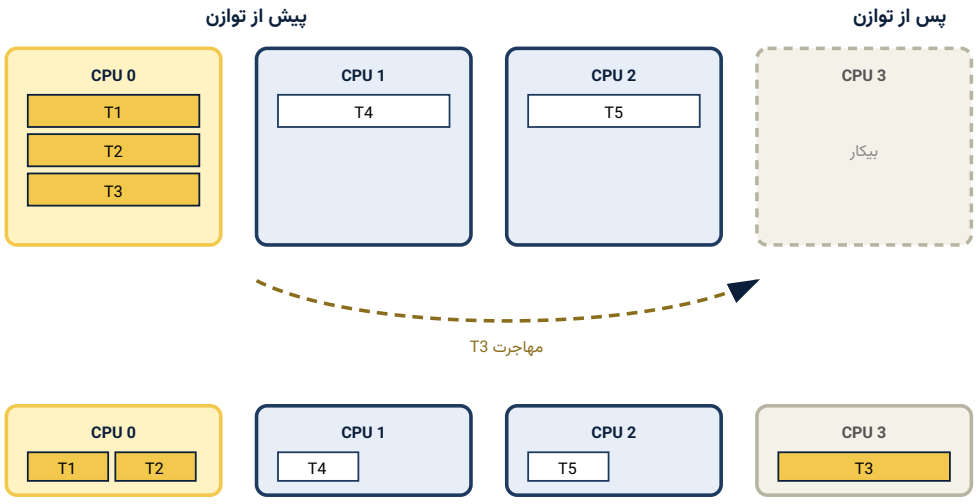
شکل ۸٫۴ سه الگوریتم را در ابعاد کیفی مقایسه می‌کند. امتیازها گویا هستند و برای پشتیبانی از بحث فصل ۹ طراحی شده‌اند، نه ارائه یک محک کمی.



شکل ۸٫۴ — مقایسه کیفی CFS، RM و EDF در انصاف، پشتیبانی از مهلت، پیش‌بینی‌پذیری، سادگی و تناسب با SMP. امتیازها گویا هستند و نباید به‌عنوان اندازه‌گیری تلقی شوند. CFS در انصاف و مقیاس‌پذیری SMP قوی است؛ RM و EDF تحت مفروضات کلاسیک خود در پشتیبانی از مهلت و پیش‌بینی‌پذیری قوی‌ترند.

۸٫۵ توازن بار میان چهار هسته

شکل ۸٫۵ یک سناریوی توازن بار را نشان می‌دهد. پیش از توازن، CPU 0 سه وظیفه و CPU 3 بیکار است؛ پس از توازن، یک وظیفه مهاجرت کرده و عدم توازن کاهش یافته است. نمودار مفهومی است و سلسله‌مراتب دامنه‌های زمان‌بندی را نادیده می‌گیرد.



شکل ۸٫۵ — توازن بار مفهومی میان چهار هسته CPU. متوازن‌کننده ممکن است وظیفه‌ای را از یک CPU به‌شدت بارگذاری‌شده به یک CPU بیکار مهاجرت دهد. توازن واقعی دامنه‌های زمان‌بندی، وابستگی حافظه نهان و توپولوژی NUMA را در نظر می‌گیرد و ممکن است وظیفه دیگری یا هیچ مهاجرتی را انتخاب نکند.

هشدار منشأ داده

نمودارهای این فصل **گویا** هستند. آن‌ها از مدل‌های ساده‌شده و وزن‌های گردشده ساخته شده‌اند، نه از داده‌های ردگیری هسته یا مجموعه‌های محک. هیچ ادعایی درباره معناداری آماری، بازه‌های اطمینان یا بازتولیدپذیری مطرح نمی‌شود. برای داده‌های عملکرد تجربی، خوانندگان باید به ادبیات پژوهش اصلی، محک‌های فهرست پستی هسته و مطالعات وابسته به بار کاری مراجعه کنند.

۹. مقایسه CFS، نرخ یکنوا (RM) و اولین مهلت (EDF)

۹٫۱ CFS: زمان‌بندی منصفانه همه‌منظوره

CFS برای بارهای کاری همه‌منظوره طراحی شده است که هدفش دادن سهم منصفانه‌ای از زمان CPU به هر وظیفه آماده اجرا متناسب با وزنش است. نیازی ندارد وظایف مهلت، دوره یا زمان اجرای بدترین حالت خود را اعلام کنند. الگوهای ورود دلخواه وظایف و رفتار انسداد را مدیریت می‌کند. نقاط قوت آن انصاف، سادگی پیکربندی و رفتار خوب تحت بارهای کاری ترکیبی است. نقطه ضعف آن این است که تضمین مهلت سخت ارائه نمی‌دهد: سهم منصفانه‌ای از زمان CPU با تضمین این‌که یک وظیفه در زمان مشخصی کامل شود یکسان نیست.

۹٫۲ RM: زمان‌بندی بی‌درنگ با اولویت ثابت

زمان‌بندی نرخ یکنوا یک الگوریتم اولویت ثابت برای وظایف دوره‌ای بی‌درنگ است. به هر وظیفه اولیتی نسبت معکوس با دوره‌اش داده می‌شود: وظایف با دوره کوتاه‌تر اولویت بالاتری دریافت می‌کنند. RM در میان الگوریتم‌های اولویت ثابت برای وظایف دوره‌ای با مهلت‌های برابر با دوره‌ها، تحت مفروضات کلاسیک وظایف مستقل، پیشگیرانه روی یک تک‌پردازنده، بهینه است. کران بهره‌وری کلاسیک لیو و لیلند عبارت است از:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \le n\left(2^{1/n} - 1\right) \tag{۹٫۱}$$

که C_i زمان اجرای بدترین حالت وظیفه i، T_i دوره آن و n تعداد وظایف است. با رشد n، کران به $\ln 2 \approx 0.693$ همگرا می‌شود. این یک شرط *کافی* زمان‌بندی‌پذیری است: اگر کل بهره‌وری در یا زیر کران باشد، مجموعه وظایف تحت RM زمان‌بندی‌پذیر است. این شرط *لازم* نیست: برخی مجموعه‌های وظایف با بهره‌وری بالاتر از کران همچنان زمان‌بندی‌پذیرند و تحلیل دقیق زمان پاسخ می‌تواند این را اثبات کند. کران مهلت‌های ضمنی، وظایف مستقل، اجرای پیشگیرانه و سربار صفر تعویض زمینه را فرض می‌کند.

۹٫۳ EDF: زمان‌بندی بی‌درنگ با اولویت پویا

اولین مهلت اولویت‌ها را به‌صورت پویا بر اساس مهلت‌های مطلق تخصیص می‌دهد: وظیفه‌ای که نزدیک‌ترین مهلت را دارد اول اجرا می‌شود. روی یک تک‌پردازنده پیشگیرانه با وظایف دوره‌ای مستقل و مهلت‌های برابر با دوره‌ها، EDF بهینه است: یک مجموعه وظایف زمان‌بندی‌پذیر است اگر و تنها اگر کل بهره‌وری آن حداکثر ۱٫۰ باشد. این نتیجه‌ای قوی‌تر از کران RM است، اما به همان مفروضات آرمانی بستگی دارد. EDF می‌تواند تحت اضافه‌بار فاجعه‌بار شکست بخورد و رفتار آن روی چندپردازنده‌ها پیچیده‌تر است. سیستم‌های عملی کنترل پذیرش، اجرای مهلت و محدودسازی را برای جلوگیری از اضافه‌بار اضافه می‌کنند.

۹٫۴ جدول مقایسه تفصیلی

جدول ۹٫۱ CFS، RM و EDF را در معیارهای لازم برای این مطالعه مقایسه می‌کند. مقایسه برجسته می‌سازد که CFS و الگوریتم‌های بی‌درنگ فضاهای طراحی متفاوتی را اشغال می‌کنند: CFS انصاف و عمومیت را بهینه می‌کند، در حالی که RM و EDF رعایت مهلت را تحت مفروضات صریح بی‌درنگ بهینه می‌کنند.

جدول ۹٫۱ — مقایسه تفصیلی CFS، RM و EDF.

معیار	CFS	نرخ یکنوا (RM)	اولین مهلت (EDF)
هدف اصلی	انصاف نسبی میان وظایف آماده اجرا	رعایت مهلت برای وظایف دوره‌ای از طریق اولویت ثابت	رعایت مهلت از طریق اولویت پویا بر اساس نزدیک‌ترین مهلت
مدل زمان‌بندی	همه‌منظوره، رویدادمحور، پیشگیرانه	مدل وظیفه دوره‌ای، پیشگیرانه، اولویت ثابت	مدل وظیفه دوره‌ای یا پراکنده، پیشگیرانه، اولویت پویا
تخصیص اولویت	وزن از مقدار nice؛ وزن‌های گروهی	دوره کوتاه‌تر → اولویت بالاتر	مهلت مطلق زودتر → اولویت بالاتر
پیش‌دستی	پیشگیرانه؛ گرانولاریتی بیدار شدن سربار را کاهش می‌دهد	پیشگیرانه	پیشگیرانه
مدیریت مهلت	بدون مفهوم مهلت	مهلت‌های ضمنی یا صریح؛ تحلیل بر اساس دوره‌ها	مهلت‌های صریح؛ نزدیک‌ترین مهلت اول
انصاف	قوی: اشتراک نسبی بر پایه زمان اجرای مجازی	هدف اصلی نیست؛ اولویت‌های ثابت می‌توانند وظایف کم‌اولویت را گرسنه بگذارند	هدف اصلی نیست؛ مهلت‌محور است
پیش‌بینی‌پذیری	آماری، وابسته به بار کاری؛ بدون کران سخت	تحلیل زمان پاسخ بدترین حالت ممکن است	تحلیل بدترین حالت تحت مفروضات ممکن است
بار کاری معمول	رومیزی، سرور، کانتینر، همه‌منظوره	کنترل دوره‌ای، نهفته بی‌درنگ	بی‌درنگ دوره‌ای/پراکنده، سیستم‌های تطبیقی
سربار زمان‌بندی	عملیات درخت O(log n)؛ قفل‌های هر CPU	کم؛ صف اولویت ثابت	متوسط؛ صف اولویت پویا
تمرکز زمان پاسخ	تأخیر میانگین و دنباله تحت رقابت	زمان پاسخ بدترین حالت	زمان پاسخ بدترین حالت
پیامدهای چندپردازنده	صف‌های هر CPU، توازن بار، وابستگی، NUMA	سراسری یا پارتیشن‌بندی؛ کران بهره‌وری تغییر می‌کند	EDF سراسری، EDF پارتیشن‌بندی و رویکردهای ترکیبی
کنترل پذیرش	ذاتی نیست؛ محدودیت پهنای باند cgroup	آزمون مبتنی بر بهره‌وری	بهره‌وری ≤ ۱ روی تک‌پردازنده؛ پیچیده‌تر روی چندپردازنده
زمان‌بندی‌پذیری نظری	کران‌های انصاف، نه تضمین مهلت	کران کافی n(2^(1/n) - 1)؛ آزمون دقیق ممکن است	بهره‌وری ≤ ۱ کافی و لازم روی تک‌پردازنده آرمانی
نقاط قوت	منصف، عمومی، مقیاس‌پذیر، آسان برای پیکربندی	ساده، پیش‌بینی‌پذیر، بهینه در میان اولویت‌های ثابت	بهینه روی تک‌پردازنده آرمانی، بهره‌وری بالاتر از RM
نقاط ضعف	بدون مهلت سخت؛ تأخیر به بار بستگی دارد	کران بهره‌وری پایین؛ انصاف میانگین ضعیف	شکننده تحت اضافه‌بار؛ پیچیدگی چندپردازنده
کاربرد معمول در سیستم‌عامل	کلاس منصفانه لینوکس (CFS/EEVDF)	RTOS، کنترل نهفته، برخی سیاست‌های RT لینوکس	SCHED_DEADLINE لینوکس، RTOS، سیستم‌های پژوهشی

۹٫۵ تحلیل زمان‌بندی‌پذیری: دامنه و محدودیت‌ها

کران RM و کران بهره‌وری EDF نتایج نظری قدرتمندی هستند، اما بر مفروضاتی تکیه دارند که در عمل به‌ندرت کاملاً برآورده می‌شوند:

- **وظایف مستقل دوره‌ای.** وظایف واقعی ممکن است وابستگی، منابع مشترک و ورودهای پراکنده داشته باشند.
- **زمان‌های اجرای بدترین حالت معلوم.** تحلیل WCET برای پردازنده‌های مدرن با حافظه‌های نهان، خطوط لوله و SMT دشوار است.
- **اجرای پیشگیرانه با سربار ناچیز.** تعویض‌های زمینه و اثرات حافظه نهان سرباری را معرفی می‌کنند که مدل آرمانی نادیده می‌گیرد.
- **زمان‌بندی تک‌پردازنده.** زمان‌بندی بی‌درنگ چندپردازنده به‌مراتب پیچیده‌تر است؛ EDF سراسری و رویکردهای پارتیشن‌بندی‌شده کران‌ها و ناهنجاری‌های متفاوتی دارند.
- **بدون اضافه‌بار.** بهینگی EDF فرض می‌کند که سیستم دچار اضافه‌بار نیست؛ تحت اضافه‌بار، مهلت‌ها می‌توانند به شیوه‌ای که پیش‌بینی آن دشوار است از دست بروند.

CFS نمی‌کوشد این مفروضات را برآورده کند و بنابراین تضمین‌های متناظر را ارائه نمی‌دهد. برعکس، RM و EDF برای ارائه انصاف نسبی میان وظایف دلخواه طراحی نشده‌اند. انتخاب میان آن‌ها به نیازهای برنامه بستگی دارد: اگر مهلت‌ها سخت و مدل وظیفه معلوم باشد، RM یا EDF با تحلیل مناسب ممکن است مناسب باشند؛ اگر بار کاری همه‌منظوره و انصاف هدف اصلی باشد، CFS (یا EEVDF) تناسب بهتری دارد.

۹٫۶ زمان تناسب هر رویکرد

CFS برای رومیزی‌ها، سرورها، کانتینرها و بارهای کاری ترکیبی مناسب است که هیچ وظیفه‌ای مهلت سخت ندارد و هدف اشتراک منصفانه و پاسخ‌گویانه CPU است. **RM** برای سیستم‌های بی‌درنگ دوره‌ای با اولویت‌های ثابت مناسب است، به‌ویژه وقتی تحلیل زمان پاسخ بدترین حالت لازم است و بهره‌وری درون کران RM است. **EDF** برای سیستم‌های بی‌درنگ که می‌توانند از اولویت‌های پویا و بهره‌وری بالاتر سود ببرند مناسب است، به شرط جلوگیری از اضافه‌بار و درک مدل وظیفه. در لینوکس، `SCHED_DEADLINE` یک سیاست مبتنی بر EDF با کنترل پذیرش فراهم می‌کند، در حالی که `SCHED_FIFO` و `SCHED_RR` سیاست‌های بی‌درنگ با اولویت ثابت فراهم می‌کنند.

۱۰. محدودیت‌ها و معایب CFS

۱۰٫۱ فقدان تضمین ذاتی مهلت بی‌درنگ سخت

CFS یک زمان‌بند انصاف‌محور است. پارامترهای مهلت را نمی‌پذیرد، کنترل پذیرش بر اساس مهلت‌ها انجام نمی‌دهد و تضمین نمی‌کند که یک وظیفه در زمان مشخصی کامل شود. تحت رقابت سنگین، یک وظیفه کلاس منصفانه ممکن است تأخیر زمان‌بندی‌ای تجربه کند که برای یک سیستم بی‌درنگ سخت غیرقابل‌قبول باشد. این یک باگ نیست؛ پیامد اهداف طراحی است. تضمین‌های بی‌درنگ در لینوکس توسط کلاس‌های زمان‌بندی بی‌درنگ و مهلت ارائه می‌شوند، نه توسط CFS.

۱۰٫۲ انصاف در برابر رعایت مهلت

انصاف و رعایت مهلت اهداف متفاوتی هستند. یک زمان‌بند می‌تواند در بلندمدت کاملاً منصف باشد و همچنان همه مهلت‌ها را از دست بدهد اگر مهلت‌ها کوتاه‌تر از بازه اشتراک منصفانه باشند. برعکس، یک زمان‌بند مهلت‌محور می‌تواند همه مهلت‌ها را رعایت کند در حالی که به‌شدت نسبت به وظایف بدون مهلت ناعادلانه است. مکانیزم زمان اجرای مجازی CFS تضمین می‌کند که هیچ وظیفه آماده اجرا از زمان CPU گرسنه نماند، اما تضمین نمی‌کند که هیچ وظیفه‌ای در زمان مشخصی کامل شود. این تمایز بنیادین است و باید در هر مقایسه CFS با RM یا EDF تأکید شود.

۱۰٫۳ تأخیر زمان‌بندی و بارهای کاری حساس به تأخیر

تأخیر زمان‌بندی CFS به تعداد وظایف آماده اجرا، تأخیر هدف و گرانولاریتی حداقل بستگی دارد. وقتی بسیاری از وظایف آماده اجرا باشند، فاصله میان اجراهای متوالی یک وظیفه مشخص می‌تواند رشد کند که برای صدا، ویدئو یا حلقه‌های کنترل غیرقابل‌قبول باشد. گرانولاریتی بیدار شدن سربار پیش‌دستی را کاهش می‌دهد اما می‌تواند یک وظیفه حساس به تأخیر تازه بیدارشده را نیز به تأخیر بیندازد. EEVDF برخی از این مسائل را با اجازه دادن به وظایف برای درخواست برش‌های کوتاه‌تر و با استفاده از مهلت‌های مجازی حل می‌کند، اما حتی EEVDF نیز یک زمان‌بند بی‌درنگ سخت نیست.

۱۰٫۴ همگام‌سازی و رقابت صف اجرا

صف‌های اجرای هر CPU مقیاس‌پذیری را بهبود می‌بخشند اما نیازمند همگام‌سازی برای مهاجرت، توازن بار و تغییرات پارامتر هستند. تحت بارهای کاری حاوی بسیاری از وظایف کوتاه‌عمر یا مهاجرت‌های مکرر، رقابت قفل می‌تواند به گلوگاه تبدیل شود. هسته این را با قفل‌بندی دقیق و RCU کاهش می‌دهد، اما رقابت را نمی‌توان کاملاً حذف کرد. این یک محدودیت در سطح پیاده‌سازی است نه مفهومی، اما بر عملکرد دنیای واقعی اثر می‌گذارد.

۱۰٫۵ سربار مهاجرت و اختلال حافظه نهان

توازن بار ممکن است وظایف را میان CPUها مهاجرت دهد که می‌تواند بهره‌وری را بهبود بخشد اما محلیت حافظه نهان را مختل کند. یک وظیفه مهاجرت‌کرده ممکن است نیاز به بارگذاری مجدد مجموعه کاری خود داشته باشد که باعث خطاهای حافظه نهان و TLB می‌شود. CFS می‌کوشد مزایای مهاجرت را با این هزینه‌ها متوازن کند، اما سازش به

بار کاری بستگی دارد. در برخی موارد، نگه داشتن یک وظیفه روی یک CPU پرمشغله با حافظه نهان گرم بهتر از مهاجرت به یک CPU بیکار با حافظه نهان سرد است.

۱۰٫۶ اثرات NUMA

در سیستم‌های NUMA، مهاجرت یک وظیفه در گره‌ها می‌تواند تأخیر حافظه را افزایش دهد اگر داده‌های وظیفه روی گره اصلی بمانند. توازن بار آگاه از NUMA در CFS محلیت را در نظر می‌گیرد، اما بهینه‌سازی کامل NUMA دشوار است زیرا به الگوهای دسترسی حافظه بستگی دارد که زمان‌بند نمی‌تواند کاملاً مشاهده کند. بارهای کاری با مجموعه‌های کاری بزرگ و الگوهای دسترسی نامنظم ممکن است عملکرد زیربهینه تجربه کنند.

۱۰٫۷ پیچیدگی‌های حسابداری انصاف

حسابداری انصاف CFS پیچیده‌تر از معادله ساده‌شده ۳٫۲ است. زمان‌بندی گروهی، ردگیری بار، جای‌گذاری در بیدار شدن و تنظیمات مهاجرت همگی بر زمان اجرای مجازی مؤثر اثر می‌گذارند. باگ‌ها یا پیکربندی‌های نادرست در این مکانیزم‌ها می‌توانند انحرافات انصاف تولید کنند. این پیچیدگی همچنین پیش‌بینی سهم‌های دقیق CPU را در پیکربندی‌های سلسله‌مراتبی برای مدیران دشوار می‌سازد.

۱۰٫۸ پیچیدگی‌های زمان‌بندی گروهی و تخصیص سلسله‌مراتبی

زمان‌بندی گروهی وزن‌های سلسله‌مراتبی، محدودیت‌های پهنای باند و محدودسازی را معرفی می‌کند. این مکانیزم‌ها قدرتمند اما پیچیده‌اند. یک cgroup با پیکربندی نادرست می‌تواند کمتر از حد مورد نظر CPU دریافت کند، یا حتی وقتی سیستم کاملاً بارگذاری نشده است محدود شود. تعامل میان وزن‌های گروهی و وزن‌های هر وظیفه همیشه شهودی نیست و اشکال‌زدایی نیازمند آشنایی با cgroup v2 و کنترل پهنای باند CFS است.

۱۰٫۹ رفتار وابسته به بار کاری

عملکرد CFS به بار کاری بستگی دارد. پیکربندی‌ای که برای یک وب‌سرور خوب کار می‌کند ممکن است برای یک مزرعه ساخت یا یک سیستم کنترل بی‌درنگ زیربهینه باشد. هیچ مجموعه پارامتر قابل تنظیم «بهترین» واحدی برای همه بارهای کاری وجود ندارد. مدیران باید بار کاری خود را درک و رفتار زمان‌بندی را برای رسیدن به نتایج خوب پایش کنند.

۱۰٫۱۰ تفاوت‌های نسخه هسته

کلاس زمان‌بندی منصفانه در طول زمان تغییر کرده است. رفتار CFS کلاسیک در لینوکس x.۵ با رفتار EEVDF در لینوکس ۶٫۶ و بعد متفاوت است. sysctlها و پارامترهای قابل تنظیم اضافه، حذف یا تغییر نام داده شده‌اند. مستندات و توصیه‌های نوشته‌شده برای یک نسخه هسته ممکن است برای نسخه دیگری نادرست باشند. این یک محدودیت عملی مهم است: رفتار زمان‌بندی را نمی‌توان بدون مشخص کردن نسخه هسته و پیکربندی درک کرد.

۱۰٫۱۱ محدودیت‌های مدل‌های نظری ساده‌شده

مدل‌های ساده‌شده مانند معادله ۳٫۲ و کران‌های RM/EDF برای درک مفیدند اما بسیاری از اثرات دنیای واقعی را نادیده می‌گیرند: رفتار حافظه نهان، رقابت SMT، پهنای باند حافظه، تداخل I/O و سربار مجازی‌سازی. نتایج حاصل از مدل‌های آرمانی باید پیش از اعمال در سیستم‌های عملی به‌صورت تجربی اعتبارسنجی شوند. مثال‌های گویای این

مقاله صراحتاً آموزشی هستند و نباید به‌عنوان پیش‌بینی عملکرد تلقی شوند.

۱۱. گرسنگی، بن‌بست و وارونگی اولویت

۱۱٫۱ گرسنگی

تعریف. گرسنگی وقتی رخ می‌دهد که یک وظیفه آماده اجرا است اما به‌طور نامحدود از پیشرفت بازداشته می‌شود زیرا سایر وظایف همیشه اول انتخاب می‌شوند. در یک زمان‌بند کاملاً منصف، گرسنگی میان وظایف کلاس منصفانه تحت شرایط عادی بعید است زیرا زمان‌بند همیشه وظیفه با کوچک‌ترین زمان اجرای مجازی را انتخاب می‌کند. وظیفه‌ای که مدتی منتظر مانده زمان اجرای مجازی نسبتاً کوچکی انباشته می‌کند، بنابراین در نهایت باید چپ‌ترین گره شود و انتخاب گردد. با این حال، گرسنگی همچنان می‌تواند به دلایل خارج از کنترل زمان‌بند منصف رخ دهد:

- **کلاس‌های زمان‌بندی با اولویت بالاتر.** یک جریان پیوسته از وظایف بی‌درنگ می‌تواند وظایف کلاس منصفانه را به‌طور نامحدود گرسنه بگذارد مگر این‌که محدودسازی بی‌درنگ پیکربندی شود.
- **محدودسازی پهنای باند CPU.** یک cgroup که از سهمیه `cpu.max` خود فراتر می‌رود ممکن است محدود شود و از اجرای وظایفش جلوگیری کند حتی اگر آماده اجرا باشند.
- **بن‌بست منابع.** وظیفه‌ای که روی یک قفل یا عملیات I/O مسدود شده نمی‌تواند پیشرفت کند فارغ از این‌که زمان CPU چقدر منصفانه توزیع شود.
- **وارونگی اولویت.** یک وظیفه با اولویت بالا ممکن است به‌طور غیرمستقیم توسط یک وظیفه با اولویت پایین‌تر که منبع مورد نیاز را در اختیار دارد به تأخیر بیفتد.

انصاف به سبک CFS خطر گرسنگی پایدار CPU را میان وظایف آماده اجرای کلاس منصفانه کاهش می‌دهد، اما گرسنگی را غیرممکن نمی‌سازد. تمایز میان گرسنگی، اجرای تأخیری، محدودسازی، انسداد و گرسنگی ناشی از سایر مکانیزم‌ها برای تشخیص درست مهم است.

۱۱٫۲ بن‌بست

تعریف. بن‌بست وضعیتی است که در آن مجموعه‌ای از وظایف هر یک منتظر منبعی هستند که در اختیار وظیفه دیگری از همان مجموعه است، بنابراین هیچ‌کدام نمی‌توانند پیشرفت کنند. بن‌بست عمدتاً یک مسئله تخصیص منابع و همگام‌سازی است، نه پیامد سیاست انتخاب CPU. چهار شرط کافمن برای بن‌بست لازم‌اند:

1. **انحصار متقابل.** حداقل یک منبع در حالتی غیرقابل اشتراک نگه داشته می‌شود.
2. **نگه‌داشتن و انتظار.** یک وظیفه حداقل یک منبع را نگه می‌دارد و منتظر منابع اضافی است که در اختیار وظایف دیگرند.
3. **بدون پیش‌دستی.** منابع نمی‌توانند به‌زور از یک وظیفه گرفته شوند؛ باید به‌طور داوطلبانه آزاد شوند.
4. **انتظار دایره‌ای.** یک زنجیره دایره‌ای از وظایف وجود دارد که هر یک منتظر منبعی است که در اختیار وظیفه بعدی است.

وظیفه‌ای که توسط بن‌بست مسدود شده نمی‌تواند پیشرفت کند فارغ از این‌که CPU چقدر منصفانه زمان‌بندی شود. CFS می‌تواند تضمین کند که وظیفه در هنگام آماده اجرا شدن زمان CPU دریافت می‌کند، اما نمی‌تواند چرخه منابع را حل کند. پیشگیری، اجتناب، تشخیص و بازیابی از بن‌بست دغدغه‌های جداگانه‌ای هستند که به زیرسیستم‌های

همگام‌سازی و مدیریت منابع تعلق دارند.

۱۱٫۳ وارونگی اولویت

تعریف. وارونگی اولویت وقتی رخ می‌دهد که یک وظیفه با اولویت بالا به‌طور غیرمستقیم توسط یک وظیفه با اولویت پایین‌تر که منبع مورد نیاز وظیفه با اولویت بالا را نگه داشته به تأخیر می‌افتد، در حالی که یک وظیفه با اولویت متوسط وظیفه با اولویت پایین‌تر را پیش‌دستی می‌کند. مثال کلاسیک شامل سه وظیفه است: H (اولویت بالا)، M (اولویت متوسط) و L (اولویت پایین). L یک قفل را کسب می‌کند. H آماده اجرا می‌شود و می‌کوشد همان قفل را کسب کند اما مسدود می‌شود زیرا L آن را در اختیار دارد. M آماده اجرا می‌شود و از آن‌جا که اولویتش از L بالاتر است، L را پیش‌دستی می‌کند. اکنون H منتظر L است اما L نمی‌تواند اجرا شود زیرا M در حال اجراست. وظیفه با اولویت بالا عملاً توسط وظیفه با اولویت متوسط به تأخیر افتاده است.

وارونگی اولویت از گرسنگی و بن‌بست متفاوت است. این ناشی از نادیده گرفتن وظیفه آماده اجرا توسط زمان‌بند CPU نیست؛ ناشی از تعامل میان زمان‌بندی مبتنی بر اولویت و قفل‌گذاری منابع است. راه‌حل‌ها شامل **ارث‌بری اولویت** (نگه‌دارنده قفل به‌طور موقت اولویت بالاترین منتظر را به ارث می‌برد) و پروتکل‌های **سقف اولویت** است. این مکانیزم‌ها به‌ویژه در سیستم‌های بی‌درنگ مهم‌اند و توسط RT-mutexها در لینوکس پشتیبانی می‌شوند. خود CFS ارث‌بری اولویت را برای وظایف کلاس منصفانه پیاده‌سازی نمی‌کند، بنابراین وارونگی اولویت شامل وظایف کلاس منصفانه باید از طریق طراحی همگام‌سازی حل شود.

۱۱٫۴ جدول مقایسه

جدول ۱۱٫۱ — گرسنگی در برابر بن‌بست در برابر وارونگی اولویت.

جنبه	گرسنگی	بن‌بست	وارونگی اولویت
مسئله اصلی	وظیفه‌ای هرگز CPU یا منابع دریافت نمی‌کند	انتظار دایره‌ای؛ هیچ وظیفه‌ای پیشرفت نمی‌کند	وظیفه با اولویت بالا توسط نگه‌دارنده قفل با اولویت پایین‌تر به تأخیر می‌افتد
علت	انتخاب ناعادلانه، کلاس‌های با اولویت بالاتر، محدودسازی، انسداد	چرخه منابع که شرایط کافمن را برآورده می‌کند	زمان‌بندی اولویت‌محور + منبع غیرقابل پیش‌دستی که در اختیار وظیفه با اولویت پایین است
رابطه با CFS	حسابداری منصفانه خطر را میان وظایف آماده اجرای کلاس منصفانه کاهش می‌دهد، اما همه علل را از بین نمی‌برد	ناشی از انتخاب CPU نیست؛ یک مسئله همگام‌سازی است	ناشی از CFS به‌تنهایی نیست؛ یک مسئله همگام‌سازی و تعامل اولویت است
نشانه معمول	وظیفه به‌طور نامحدود منتظر می‌ماند؛ زمان CPU کم	سیستم یا مجموعه وظایف معلق می‌شود؛ بدون پیشرفت	وظیفه با اولویت بالا با وجود آماده اجرا بودن مهلت را از دست می‌دهد
کاهش	زمان‌بندی منصفانه، محدودسازی بی‌درنگ، محدودیت‌های cgroup، رفع گلوگاه‌های منابع	پیشگیری، اجتناب، تشخیص و بازیابی بن‌بست	ارث‌بری اولویت، سقف اولویت، طراحی دقیق قفل

۱۲. امنیت، قابلیت اطمینان و ملاحظات عملی

۱۲٫۱ مرزهای ایزولاسیون زمان‌بند

زمان‌بند بخشی از پایگاه محاسباتی مورد اعتماد هسته است. یک باگ در کد زمان‌بندی می‌تواند باعث خرابی، فساد داده یا انکار سرویس شود. CFS و زیرساخت پشتیبان آن بنابراین تابع استانداردهای سخت کدنویسی و بازبینی هسته هستند. رابط‌های فضای کاربر مانند nice، وابستگی و کنترل‌های cgroup با بررسی‌های مجوز وساطت می‌شوند. کاربران غیرمجاز می‌توانند مقادیر nice را در محدوده‌های مجاز تغییر دهند و وابستگی وظایف خود را تنظیم کنند، اما نمی‌توانند سیاست‌های زمان‌بندی یا وظایف سایر کاربران را بدون مجوزهای مناسب به‌طور دلخواه تغییر دهند.

۱۲٫۲ تخلیه منابع و ملاحظات انکار سرویس

یک وظیفه مخرب یا دارای باگ که بسیاری از رشته‌های آماده اجرا را ایجاد می‌کند می‌تواند سربار زمان‌بندی را تحمیل و پاسخ‌دهی سیستم را تخریب کند. صف‌های اجرای هر CPU و عملیات O(log n) در CFS هزینه تصمیم‌های زمان‌بندی منفرد را محدود می‌کنند، اما هزینه تجمیعی بسیاری از وظایف همچنان می‌تواند قابل توجه باشد. سهمیه‌های CPU و محدودیت‌های پهنای باند cgroup می‌توانند اثر یک گروه سرکش را مهار کنند. محدودسازی بی‌درنگ مانع مصرف ۱۰۰٪ زمان CPU به‌طور نامحدود توسط وظایف بی‌درنگ می‌شود. این مکانیزم‌ها تضمین امنیت مطلق نیستند؛ آن‌ها ابزارهای مدیریت منابع هستند که باید به‌درستی پیکربندی شوند.

۱۲٫۳ سهمیه‌های CPU و کنترل‌های منابع مبتنی بر cgroup

کنترل‌کننده CPU در cgroup v2 دو مکانیزم اصلی ارائه می‌دهد: **وزن** (اشتراک نسبی) و **پهنای باند** (محدودیت مطلق). وزن از طریق `cpu.weight` تنظیم می‌شود و بر سهم گروه در زمان رقابت CPU اثر می‌گذارد. پهنای باند از طریق `cpu.max` تنظیم می‌شود که یک سهمیه و دوره را مشخص می‌کند؛ گروهی که از سهمیه خود فراتر می‌رود تا دوره بعد محدود می‌شود. این کنترل‌ها در محیط‌های کانتینری که چندین مستأجر یک میزبان را به اشتراک می‌گذارند ضروری هستند. پیکربندی نادرست می‌تواند به محدودسازی حتی وقتی میزبان به‌طور کامل بهره‌ور نیست منجر شود، بنابراین پایش و تنظیم مهم هستند.

۱۲٫۴ پیکربندی سیاست زمان‌بندی

لینوکس سیاست زمان‌بندی را از طریق `sched_setscheduler(2)`، `sched_setattr(2)` و ابزارهای خط فرمان در معرض دید قرار می‌دهد. سیاست‌های منصفانه `SCHED_NORMAL`، `SCHED_BATCH` و `SCHED_IDLE` هستند. سیاست‌های بی‌درنگ `SCHED_FIFO` و `SCHED_RR` هستند و زمان‌بندی مهلت از `SCHED_DEADLINE` استفاده می‌کند. تغییر سیاست نیازمند مجوزهای مناسب (به‌طور معمول `CAP_SYS_NICE`) است. مدیران باید محتاط باشند: تخصیص `SCHED_FIFO` به یک وظیفه پرمشغله می‌تواند وظایف کلاس منصفانه را گرسنه بگذارد و سیستم را غیرپاسخگو کند.

۱۲٫۵ سازش‌های تنظیم هسته

CFS چندین پارامتر قابل تنظیم در معرض دید قرار می‌دهد، شامل `sched_latency_ns`، `sched_min_granularity_ns` و `sched_wakeup_granularity_ns`. کاهش تأخیر پاسخ‌دهی را بهبود می‌بخشد اما سربار تعویض زمینه را افزایش می‌دهد. افزایش آن گذردهی را برای بارهای کاری CPU-محور بهبود می‌بخشد اما می‌تواند به عملکرد تعاملی آسیب بزند. مقادیر بهینه به بار کاری و سخت‌افزار بستگی دارند. نسخه‌های هسته ممکن است پیش‌فرض‌ها را تغییر دهند یا پارامترهای قابل تنظیم را حذف کنند، به‌ویژه با گذار به EEVDF، بنابراین توصیه‌های تنظیم باید مختص نسخه باشند.

۱۲٫۶ پایش بهره‌وری CPU و تأخیر

ابزارهایی مانند `top`، `htop`، `pidstat` و `perf sched` می‌توانند بهره‌وری CPU، طول صف اجرا، نرخ تعویض زمینه و تأخیر زمان‌بندی را آشکار کنند. `/proc/sched` آمار زمان‌بندی هر وظیفه را در معرض دید قرار می‌دهد. پایش برای تشخیص گرسنگی، مهاجرت‌های بیش‌ازحد و محدودسازی ضروری است. با این حال، تفسیر این ابزارها نیازمند درک نسخه و پیکربندی زمان‌بند است؛ معیاری که در یک هسته نشانه مشکل است ممکن است در هسته دیگری عادی باشد.

۱۲٫۷ قابلیت اطمینان در محیط‌های رومیزی، سرور و کانتینر

CFS به‌طور گسترده در رومیزی‌ها، سرورها و سیستم‌های نهفته مستقر شده است. انصاف و مقیاس‌پذیری آن را برای بارهای کاری ترکیبی مناسب می‌کند و یکپارچگی‌اش با cgroups آن را در هماهنگ‌کننده‌های کانتینر قابل استفاده می‌سازد. دغدغه‌های قابلیت اطمینان با محیط متفاوت‌اند: رومیزی‌ها پاسخ‌دهی را در اولویت قرار می‌دهند، سرورها گذردهی و تأخیر دنباله را، و کانتینرها ایزولاسیون و تخصیص پیش‌بینی‌پذیر را. هیچ پیکربندی واحدی برای هر سه بهینه نیست. معرفی EEVDF بازتاب تلاش جاری برای بهبود تأخیر بدون فدا کردن انصاف است، به‌ویژه برای وظایف حساس به تأخیر در محیط‌های همه‌منظوره.

۱۳. نمایش عملی در لینوکس

این فصل یک نمایش آموزشی قابل بازتولید ارائه می‌دهد. دستورها غیرمخرب هستند و روی یک سیستم لینوکس معمولی اجرا می‌شوند. برخی عملیات نیازمند مجوز ریشه یا CAP_SYS_NICE هستند؛ این موارد ذکر شده‌اند. هدف مشاهده رفتار زمان‌بندی است، نه محک زدن هسته.

۱۳٫۱ بررسی فرایندها با ps

```
ps -eo pid,ni,pri,psr,pcpu,comm --sort=-pcpu | head -20
```

ستون ni مقدار nice، ستون pri اولویت هسته، ستون psr شماره CPUی که فرایند آخرین بار روی آن اجرا شده و ستون pcpu بهره‌وری CPU را نشان می‌دهد. این دستور یک تصویر لحظه‌ای از این‌که کدام وظایف CPU مصرف می‌کنند و در چه سطح nice هستند ارائه می‌دهد. تأخیر زمان‌بندی یا طول صف اجرا را نشان نمی‌دهد.

۱۳٫۲ مشاهده رفتار صف اجرا با top یا htop

```
top -o %CPU
# یا
htop
```

top میانگین بار، بهره‌وری CPU و مصرف CPU هر فرایند را نمایش می‌دهد. میانگین بار شامل وظایف آماده اجرا و وظایف غیرقابل وقفه است، بنابراین معیار خالص طول صف اجرا نیست. htop نمای تعاملی‌تری ارائه می‌دهد و می‌تواند بهره‌وری هر CPU را نشان دهد. این ابزارها کمک می‌کنند تشخیص دهید آیا سیستم اشباع CPU است، اما به‌طور مستقیم vruntime یا تأخیر زمان‌بندی را در معرض دید قرار نمی‌دهند.

۱۳٫۳ تغییر مقادیر nice با nice و renice

```
# اجرای یک فرایند CPU-محور با nice 10 (اولویت پایین‌تر)
nice -n 10 sha256sum /dev/zero

# تغییر مقدار nice (ممکن است نیازمند مجوز باشد) یک فرایند موجود
renice -n 5 -p <PID>

# بررسی مقدار nice
ps -o pid,ni,comm -p <PID>
```

تغییر مقدار nice وزن وظیفه و بنابراین سهم نسبی CPU آن را تغییر می‌دهد. این کار کلاس زمان‌بندی را تغییر نمی‌دهد. یک وظیفه با nice ۱۹ همچنان یک وظیفه کلاس منصفانه است و می‌تواند توسط وظایف بی‌درنگ پیش‌دستی شود. مقادیر nice منفی نیازمند مجوز هستند و در صورت استفاده نادرست می‌توانند وظایف دیگر را گرسنه

بگذارند.

۱۳٫۴ تنظیم وابستگی CPU با taskset

```
# اجرای یک دستور سنجاق‌شده به CPU 0
taskset -c 0 sha256sum /dev/zero

# تغییر وابستگی یک فرایند موجود (ممکن است نیازمند مجوز باشد)
taskset -pc 1 <PID>

# بررسی وابستگی
taskset -pc <PID>
```

taskset ماسک وابستگی CPU را دستکاری می‌کند. سنجاق کردن یک وظیفه به یک CPU می‌تواند محلیت حافظه نهان را بهبود بخشد اما ممکن است سایر CPUها را بیکار بگذارد. وابستگی یک محدودیت سخت است که توسط متوازن‌کننده بار CFS رعایت می‌شود، بنابراین باید با احتیاط استفاده شود.

۱۳٫۵ خواندن اطلاعات زمان‌بندی از /proc

```
# آمار زمان‌بندی هر وظیفه
cat /proc/<PID>/sched

# آمار سراسری زمان‌بند
cat /proc/schedstat

# تنظیمات کنترل‌کننده CPU cgroup (cgroup v2)
cat /sys/fs/cgroup/<group>/cpu.weight
cat /sys/fs/cgroup/<group>/cpu.max
```

/proc//sched فیلدهایی مانند se.sum_exec_runtime، se.vruntime و nr_migrations ارائه می‌دهد که می‌توانند به تشخیص رفتار زمان‌بندی کمک کنند. فیلدهای دقیق به نسخه هسته بستگی دارند. /proc/schedstat آمار زمان‌بندی هر CPU را ارائه می‌دهد.

۱۳٫۶ سیاست‌های بی‌درنگ با chrt

```
# نمایش سیاست زمان‌بندی و اولویت یک فرایند
chrt -p <PID>

# اجرای یک دستور با SCHED_FIFO (نیازمند مجوز) 50 و اولویت
sudo chrt -f 50 <command>

# اجرای یک دستور با SCHED_RR (نیازمند مجوز) 20 و اولویت
sudo chrt -r 20 <command>
```

`chrt` سیاست زمان‌بندی و اولویت بی‌درنگ را تغییر می‌دهد. وظایف `SCHED_FIFO` تا زمانی که مسدود یا واگذار نشوند اجرا می‌شوند؛ وظایف `SCHED_RR` از یک برش زمانی استفاده می‌کنند. این سیاست‌ها می‌توانند وظایف کلاس منصفانه را گرسنه بگذارند، بنابراین باید تنها در صورت لزوم و با محدودسازی بی‌درنگ پیکربندی‌شده استفاده شوند. تغییر مقدار nice یک وظیفه اساساً با تغییر کلاس زمان‌بندی آن متفاوت است: nice بر وزن کلاس منصفانه اثر می‌گذارد، در حالی که `chrt` می‌تواند وظیفه را به کلاس بی‌درنگ منتقل کند.

۱۳٫۷ تفسیر خروجی و ایمنی

هنگام تفسیر خروجی زمان‌بندی، به یاد داشته باشید که نسخه هسته مهم است. سیستمی که لینوکس ۶٫۶ یا بعد را اجرا می‌کند از EEVDF در کلاس منصفانه استفاده می‌کند، بنابراین رفتار انتخاب و برخی پارامترهای قابل تنظیم با CFS کلاسیک متفاوت است. برنامه‌های آزمایشی CPU-محور را برای مدت طولانی روی سیستم‌های عملیاتی اجرا نکنید و به فرایندهای غیرقابل اعتماد اولویت بی‌درنگ تخصیص ندهید. دستورهای بالا برای آزمایش‌های کوتاه و کنترل‌شده روی یک ماشین آزمایشی یا یک ماشین مجازی یک‌بارمصرف در نظر گرفته شده‌اند.

جدول ۱۳٫۱ — ابزارهای زمان‌بندی لینوکس و کاربردهای آن‌ها.

ابزار	کاربرد	مجوزها / یادداشت
`ps`	تصویر لحظه‌ای از فرایندها، مقادیر nice، مصرف CPU	بدون مجوز ویژه
`htop` / `top`	پایش تعاملی CPU و فرایندها	بدون مجوز ویژه
`nice`	اجرای یک فرایند با مقدار nice مشخص	مقادیر منفی نیازمند مجوز
`renice`	تغییر مقدار nice یک فرایند در حال اجرا	ممکن است نیازمند مجوز
`taskset`	دریافت یا تنظیم وابستگی CPU	تغییر وابستگی وظایف سایر کاربران ممکن است نیازمند مجوز باشد
`/proc/<pid>/sched`	آمار زمان‌بندی هر وظیفه	قابل خواندن توسط مالک یا ریشه
`/proc/schedstat`	آمار سراسری زمان‌بندی	قابل خواندن توسط ریشه یا کاربران مجاز
`chrt`	دریافت یا تنظیم سیاست زمان‌بندی و اولویت بی‌درنگ	سیاست‌های بی‌درنگ نیازمند مجوز
`cpu.weight` / `cpu.max`	وزن و کنترل پهنای باند CPU در cgroup v2	نیازمند تفویض cgroup یا ریشه

۱۴. بحث و جهت‌های آینده

۱۴٫۱ تحول زمان‌بندی لینوکس

زمان‌بندی لینوکس از آرایه‌های اولویت ساده به زمان‌بندی سهم منصفانه و اخیراً به سمت زمان‌بندی منصفانه آگاه از مهلت حرکت کرده است. زمان‌بند O(1) انتخاب با زمان ثابت را در اولویت قرار می‌داد اما بر اکتشافی‌هایی برای استنتاج تعاملی‌بودن تکیه داشت. CFS این اکتشافی‌ها را با یک مدل ساعت مجازی جایگزین کرد که انصاف را صریح و مقیاس‌پذیر ساخت. EEVDF پایه اشتراک نسبی CFS را حفظ می‌کند اما تأخیر و مهلت‌های مجازی را برای بهبود کنترل تأخیر و کاهش وابستگی به پارامترهای قابل تنظیم می‌افزاید. این تحول بازتاب یک روند گسترده‌تر است: زمان‌بندهای همه‌منظوره ایده‌هایی از زمان‌بندی بی‌درنگ را برای بهبود پاسخ‌دهی بدون رها کردن انصاف در خود می‌گنجانند.

۱۴٫۲ گذار از CFS به EEVDF

گذار به EEVDF مهم است زیرا الگوریتم انتخاب کلاس منصفانه را تغییر می‌دهد و چندین اکتشافی CFS را حذف می‌کند. EEVDF به وظایف اجازه می‌دهد برش‌های زمانی مشخصی را از طریق `sched_setattr(2)` درخواست کنند و از مهلت‌های مجازی برای اولویت‌دهی به وظایف حساس به تأخیر استفاده می‌کند. همچنین از مکانیزم تأخیر پوسیده برای جلوگیری از سوءاستفاده وظایف خواب‌رفته استفاده می‌کند. گذار در لینوکس ۶٫۶ آغاز و در هسته‌های بعدی ادامه یافت. مدیران و توسعه‌دهندگان باید پیش از اعمال توصیه‌های تنظیم نوشته‌شده برای CFS کلاسیک، بررسی کنند که هسته آن‌ها از کدام الگوریتم استفاده می‌کند.

۱۴٫۳ چرا نسخه هسته مهم است

رفتار زمان‌بندی یک ویژگی «لینوکس» به‌طور انتزاعی نیست؛ یک ویژگی یک نسخه هسته و پیکربندی مشخص است. کلاس منصفانه در ۶٫۶ تغییر کرد، رابط‌های cgroup تکامل یافتند و پارامترهای قابل تنظیم اضافه یا حذف شدند. مقاله‌ای که CFS را بدون مشخص کردن نسخه هسته توصیف می‌کند خطر درهم‌آمیختن CFS کلاسیک، EEVDF و حالت‌های میانی را دارد. این مقاله کوشیده است درباره گزاره‌های وابسته به نسخه صریح باشد، اما خوانندگان باید برای هسته دقیق خود به مستندات مراجعه کنند.

۱۴٫۴ توازن انصاف، تأخیر، گذردهی و مقیاس‌پذیری

چالش محوری زمان‌بندی همه‌منظوره توازن اهداف رقیب است. انصاف نیازمند دادن نوبت به هر وظیفه آماده اجراست؛ تأخیر نیازمند اولویت دادن به برخی وظایف است؛ گذردهی اجراهای طولانی و محلیت حافظه نهان را ترجیح می‌دهد؛ مقیاس‌پذیری نیازمند ساختارهای هر CPU و کمینه همگام‌سازی میان CPUهاست. CFS یک سازش خاص انجام داد: انصاف و مقیاس‌پذیری را در اولویت قرار داد، کنترل تأخیر اکتشافی را پذیرفت و برای تضمین‌های بی‌درنگ به کلاس‌های زمان‌بندی تکیه کرد. EEVDF سازش را با معرفی پارامترهای صریح تأخیر و انتخاب مبتنی بر مهلت تنظیم می‌کند. هیچ‌کدام به‌طور جهانی بهینه نیستند؛ بهترین انتخاب به بار کاری و نیازهای سیستم بستگی دارد.

۱۴٫۵ زمان‌بندی در محیط‌های مجازی و کانتینری

مجازی‌سازی و کانتینرها ابعاد جدیدی به زمان‌بندی می‌افزایند. زمان‌بند مهمان یک ماشین مجازی روی CPUهای مجازی عمل می‌کند که خود توسط میزبان زمان‌بندی می‌شوند. وظایف یک کانتینر توسط هسته میزبان زمان‌بندی می‌شوند اما با وزن‌ها و سهمیه‌های cgroup محدود می‌گردند. این لایه‌ها می‌توانند به شیوه‌های پیچیده تعامل کنند: یک مهمان ممکن است باور کند چهار CPU دارد در حالی که میزبان CPUهای مجازی آن را روی دو هسته فیزیکی زمان‌بندی می‌کند؛ یک کانتینر ممکن است توسط `cpu.max` محدود شود حتی اگر میزبان اشباع نباشد. زمان‌بندی گروهی و کنترل پهنای باند در این محیط‌ها ابزارهای ضروری هستند اما نیازمند پیکربندی و پایش دقیق‌اند.

۱۴٫۶ جهت‌های بالقوه پژوهش آینده در زمان‌بندی

چندین جهت حوزه‌های فعال پژوهش و توسعه هستند. نخست، زمان‌بندی منصفانه آگاه از تأخیر — که EEVDF نمونه آن است — احتمالاً با مکانیزم‌های بهتر برای مشخص کردن و اجرای الزامات تأخیر بدون به خطر انداختن انصاف تکامل خواهد یافت. دوم، معماری‌های CPU ناهمگن (big.LITTLE، هسته‌های عملکردی/بازدهی) نیازمند زمان‌بندهایی هستند که تفاوت‌های ظرفیت و سازش‌های انرژی را درک کنند. سوم، اثرات سمت حافظه — محلیت NUMA، پارتیشن‌بندی حافظه نهان، پهنای باند حافظه — با رشد تعداد هسته‌ها به‌طور فزاینده مهم‌اند. چهارم، زمان‌بندی برای شتاب‌دهنده‌ها و محاسبات ناهمگن یک مسئله باز است. پنجم، تأیید رسمی و آزمون ناورداهای زمان‌بند می‌تواند قابلیت اطمینان را بهبود بخشد. این جهت‌ها حدسی هستند و باید از واقعیت‌های پیاده‌سازی مستقر تفکیک شوند.

۱۵. نتیجه‌گیری

زمان‌بند کاملاً منصفانه یکی از تأثیرگذارترین زمان‌بندهای CPU همه‌منظوره است که تاکنون مستقر شده است. ایده محوری آن — مدل کردن یک CPU ایده‌آل چندوظیفگی روی سخت‌افزار واقعی و ردگیری زمان اجرای مجازی هر وظیفه نسبت به وزنش — ساده، ظریف و مؤثر است. CFS کلاسیک با انتخاب موجودیت آماده اجرا با کوچک‌ترین زمان اجرای مجازی، انصاف نسبی را بدون نیاز به طبقه‌بندی وظایف به تعاملی یا CPU-محور فراهم می‌کند. درخت قرمز-سیاه درج و حذف کارآمد $O(\log n)$ ارائه می‌دهد، در حالی که صف‌های اجرای هر CPU و دامنه‌های زمان‌بندی از چندپردازشی مقیاس‌پذیر پشتیبانی می‌کنند.

CFS با این حال یک زمان‌بند بی‌درنگ نیست. مهلت‌ها را نمی‌پذیرد، کنترل پذیرش مبتنی بر مهلت انجام نمی‌دهد و تأخیر بدترین حالت کران‌دار را تضمین نمی‌کند. انصاف آن یک ویژگی بلندمدت است که می‌تواند در بازه‌های کوتاه منحرف شود و رفتار آن به نسخه هسته، پیکربندی و بار کاری بستگی دارد. مقایسه با نرخ یکنوا و اولین مهلت در فصل ۹ نشان داد که CFS، RM و EDF فضاهای طراحی متفاوتی را اشغال می‌کنند: CFS انصاف و عمومیت را بهینه می‌کند، RM تضمین‌های بی‌درنگ با اولویت ثابت تحت مفروضات کلاسیک ارائه می‌دهد و EDF بهینگی با اولویت پویا روی تک‌پردازنده‌های آرمانی فراهم می‌کند. هیچ‌کدام به‌طور جهانی برتر نیستند؛ انتخاب مناسب به نیازهای برنامه بستگی دارد.

گذار از CFS کلاسیک به EEVDF در لینوکس ۶٫۶ گام دیگری در تحول زمان‌بندی لینوکس است. EEVDF انصاف نسبی را حفظ می‌کند اما تأخیر و مهلت‌های مجازی را برای بهبود کنترل تأخیر و کاهش تنظیم اکتشافی می‌افزاید. این گذار اهمیت آگاهی از نسخه هسته را تقویت می‌کند: ادعاها درباره «زمان‌بند لینوکس» باید با نسخه هسته و پیکربندی محدود شوند.

در خلاصه، CFS نشان داد که انصاف می‌تواند یک هدف زمان‌بندی درجه‌یک در یک سیستم‌عامل همه‌منظوره باشد و پایه‌ای برای زمان‌بندی منصفانه آگاه از مهلت که پس از آن آمد فراهم کرد. نقاط قوت آن — اشتراک نسبی، مقیاس‌پذیری و یکپارچگی با سلسله‌مراتب کلاس‌های زمان‌بندی لینوکس — قابل توجه هستند. محدودیت‌های آن — فقدان تضمین بی‌درنگ سخت، تأخیر وابسته به بار کاری و پیچیدگی پیاده‌سازی — به همان اندازه برای درک مهم‌اند. یک ارزیابی متوازن CFS را به‌عنوان یک دستاورد مهندسی عمده که ایده‌هایش همچنان زمان‌بندی لینوکس را شکل می‌دهد به رسمیت می‌شناسد، در حالی که اذعان می‌کند هیچ الگوریتم واحدی نمی‌تواند همه اهداف زمان‌بندی را هم‌زمان برآورده کند.

جدول ۱۵٫۱ — خلاصه یافته‌های اصلی.

یافته	خلاصه
مدل CFS	یک CPU ایده‌آل چندوظیفگی را با استفاده از زمان اجرای مجازی وزن‌دار و یک درخت قرمز-سیاه تقریب می‌زند.
قاعده انتخاب	CFS کلاسیک موجودیت با کوچک‌ترین `vruntime` را انتخاب می‌کند؛ EEVDF موجودیت واجد شرایط با نزدیک‌ترین مهلت مجازی را.
انصاف	سهم‌های بلندمدت CPU با وزن‌ها متناسب‌اند؛ انحرافات کوتاه‌مدت مورد انتظارند.
چند هسته‌ای	صف‌های اجرای هر CPU، توازن بار، وابستگی و دامنه‌های زمان‌بندی آگاه از NUMA.
بی‌درنگ	CFS تضمین مهلت سخت ارائه نمی‌دهد؛ کلاس‌های بی‌درنگ و مهلت برای این هدف وجود دارند.
RM در برابر EDF	RM اولویت ثابت با کران بهره‌وری کافی است؛ EDF اولویت پویا و بهینه روی تک‌پردازنده‌های آرمانی است.
آگاهی از نسخه	لینوکس ۶٫۶ انتخاب CFS کلاسیک را با EEVDF در کلاس منصفانه جایگزین کرد؛ تنظیم و رفتار با نسخه متفاوت‌اند.
محدودیت‌ها	بدون مهلت سخت، تأخیر وابسته به بار کاری، سربار مهاجرت/حافظه نهان و پیچیدگی پیکربندی.
کاربرد عملی	CFS/EEVDF برای رومیزی‌ها، سرورها و کانتینرها مناسب هستند؛ بارهای کاری بی‌درنگ نیازمند کلاس‌های زمان‌بندی و تحلیل مناسب‌اند.

منابع

1. Linux Kernel Documentation, “CFS Scheduler,” *Documentation/scheduler/sched-design-CFS.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/sched-design-CFS.html

2. Linux Kernel Documentation, “EEVDF Scheduler,” *Documentation/scheduler/sched-eevdf.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/sched-eevdf.html

3. KernelNewbies, “Linux 6.6,” 29 October 2023. [برخط]. موجود در: https://kernelnewbies.org/Linux_6.6

4. C. L. Liu and J. W. Layland, “Scheduling Algorithms for Multiprogramming in a Hard-Real-Time Environment,” *Journal of the ACM*, vol. 20, no. 1, pp. 46–61, Jan. 1973, doi: 10.1145/321738.321743.

5. I. Stoica and H. Abdel-Wahab, “Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation,” Technical Report TR-95-22, Department of Computer Science, Old Dominion University, 1995.

6. Linux man-pages project, *sched(7) — overview of CPU scheduling*. [برخط]. موجود در: https://man7.org/linux/man-pages/man7/sched.7.html

7. Linux Kernel Documentation, “Control Group v2,” *Documentation/admin-guide/cgroup-v2.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html

8. Linux Kernel Documentation, “CFS Bandwidth Control,” *Documentation/scheduler/sched-bwc.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/sched-bwc.html

9. Linux man-pages project, *nice(1)*, *renice(1)*, *taskset(1)*, *chrt(1)*. [برخط]. موجود در: https://man7.org/linux/man-pages/

10. J. Corbet, “An EEVDF CPU scheduler for Linux,” *LWN.net*, 2023. [برخط]. موجود در: https://lwn.net/Articles/925371/

11. Linux kernel source tree, `kernel/sched/fair.c` and `kernel/sched/core.c`. [برخط]. موجود در: https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/

12. R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

13. D. P. Bovet and M. Cesati, *Understanding the Linux Kernel*, 3rd ed. Sebastopol, CA, USA: O’Reilly Media, 2005.

14. A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

15. P. Zijlstra, “EEVDF scheduler,” Linux Kernel Mailing List, 2023. [برخط]. موجود در: https://lore.kernel.org/lkml/

16. Linux Kernel Documentation, “Scheduler Statistics,” *Documentation/scheduler/sched-stats.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/

17. Linux Kernel Documentation, “Per-Entity Load Tracking,” *Documentation/scheduler/sched-pelt.c* and related kernel documentation. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/

18. M. Tim Jones, “Inside the Linux 2.6 Completely Fair Scheduler,” IBM developerWorks, 2009. [برخط]. موجود در: https://developer.ibm.com/articles/l-cfs/

19. Linux Kernel Documentation, “Real-Time Group Scheduling,” *Documentation/scheduler/sched-rt-group.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/sched-rt-group.html

20. Linux Kernel Documentation, “Deadline Task Scheduling,” *Documentation/scheduler/sched-deadline.rst*. [برخط]. موجود در: https://www.kernel.org/doc/html/latest/scheduler/sched-deadline.html